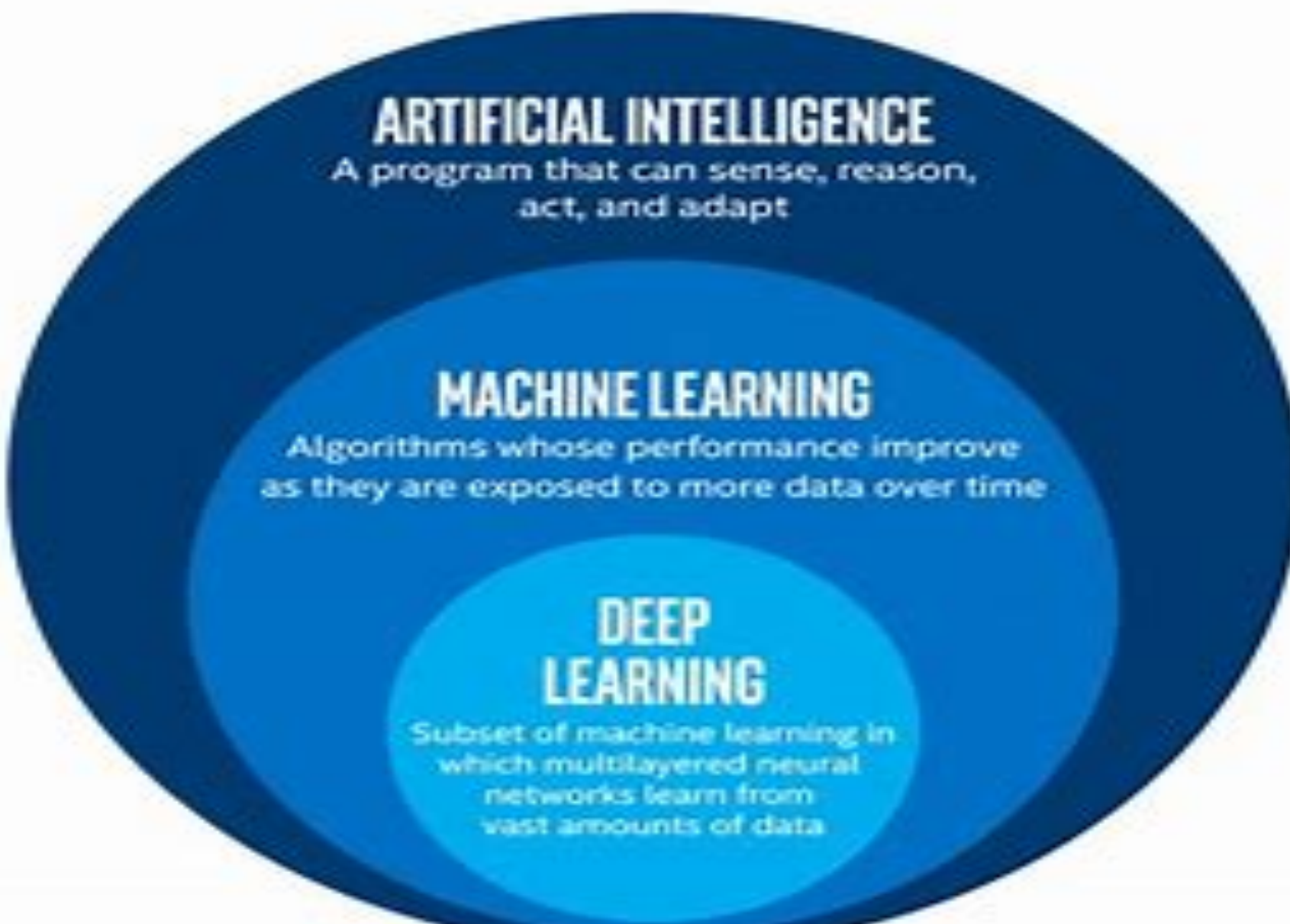


Machine Learning – Unit 1



The diagram consists of three concentric circles. The outermost circle is dark blue and contains the text 'ARTIFICIAL INTELLIGENCE' and its definition. The middle circle is a medium blue and contains the text 'MACHINE LEARNING' and its definition. The innermost circle is a light blue and contains the text 'DEEP LEARNING' and its definition. This visual structure indicates that Deep Learning is a subset of Machine Learning, which is a subset of Artificial Intelligence.

ARTIFICIAL INTELLIGENCE

A program that can sense, reason,
act, and adapt

MACHINE LEARNING

Algorithms whose performance improve
as they are exposed to more data over time

DEEP LEARNING

Subset of machine learning in
which multilayered neural
networks learn from
vast amounts of data

The Machine Learning Landscape

What Is Machine Learning?

Machine Learning is **the science (and art) of programming computers so they can learn from data.**

Slightly more general definition:

[Machine Learning is the] field of **study that gives computers the ability to learn without being explicitly programmed.** —Arthur Samuel, 1959

More engineering-oriented one:

A computer program is said **to learn from experience E with respect to some task T and some performance measure P, if its performance on T, as measured by P, improves with experience E.** —Tom Mitchell, 1997

Example:

Spam filter is a Machine Learning program that, **given examples of spam emails** (e.g., flagged by users) and **examples of regular** (nonspam, also called “ham”) **emails, can learn to flag spam.**

The **examples that the system uses to learn** are called the **training set**.
Each **training example** is called **a training instance** (or sample).

In this case, the **task T** --- is to **flag spam for new emails**,
experience E -- is the **training data**, and
the **performance measure P** --needs to be defined; -- can use **the ratio of correctly classified emails**.

This **particular performance measure** is called --**accuracy**, and it is often used in classification tasks

Why Use Machine Learning?

Eg 1 : Spam Filter

Write a spam filter using traditional programming techniques

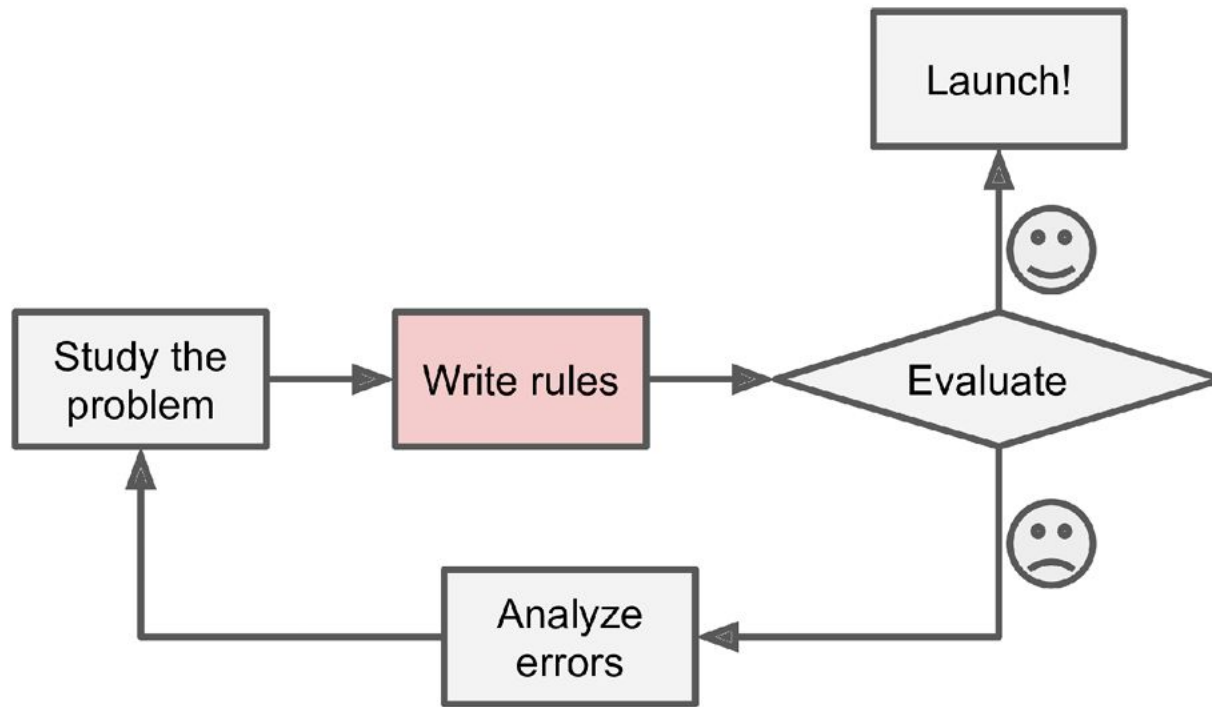


Figure 1-1. The traditional approach

1. First you would **consider what spam typically looks like.**
 - Some **words or phrases** - such as **“4U,” “credit card,” “free,”** and **“amazing”** tend to come up a lot in the subject line.
 - Also notice a few **other patterns in the sender’s name, the email’s body, and other parts of the email.**
2. You would **write a detection algorithm for each of the patterns noticed,** and your program would flag emails as spam if a number of these patterns were detected.
3. You would **test your program and repeat steps 1 and 2 until it was good enough to launch**

Since the problem is difficult, **your program will likely become a long list of complex rules—pretty hard to maintain.**

In contrast, a spam filter based on **Machine Learning** techniques automatically learns which words and phrases are good predictors of spam by **detecting unusually frequent patterns of words in the spam examples** compared to the ham examples.

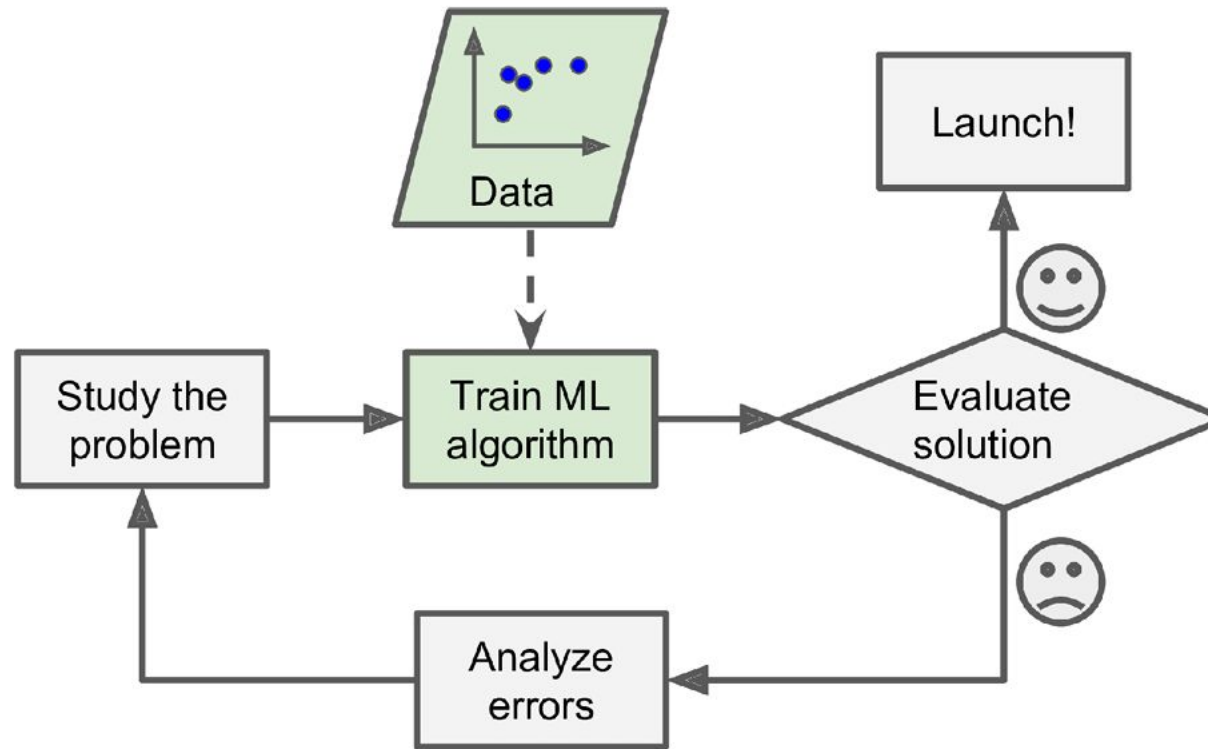


Figure 1-2. The Machine Learning approach

The program is much shorter, **easier to maintain**, and most likely more **accurate**

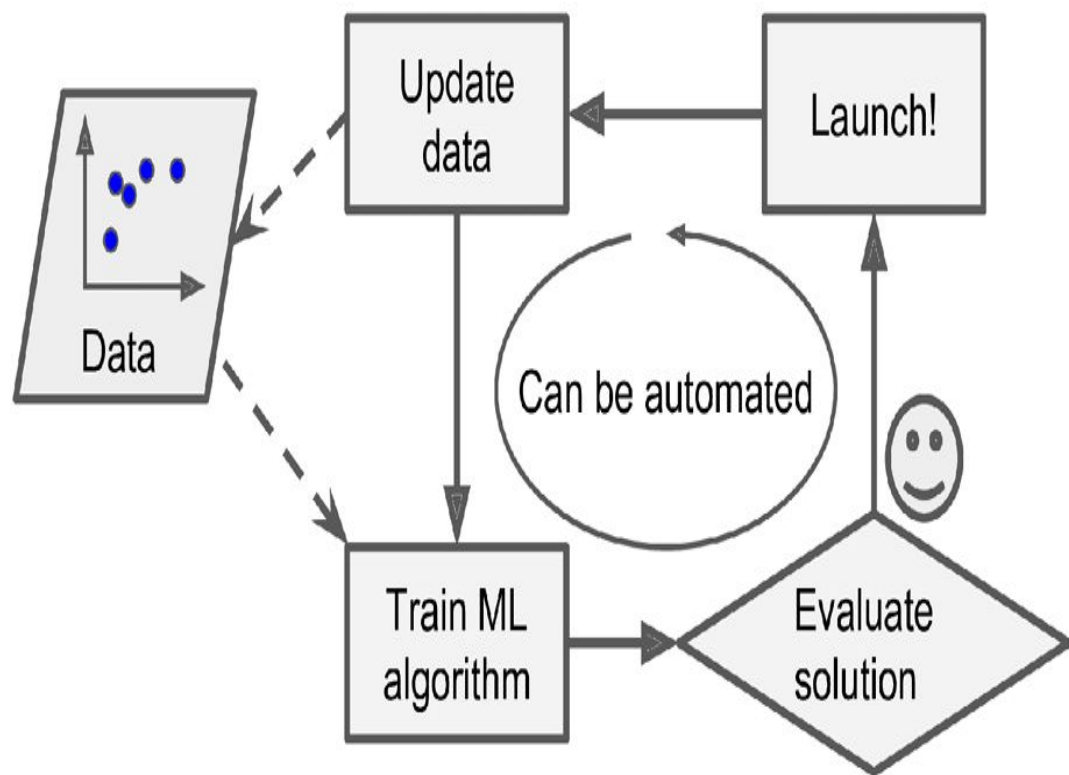


Figure 1-3. Automatically adapting to change

What if **spammers** notice that all their emails containing **“4U”** are **blocked**?

They might start writing **“For U”** instead.

A spam filter using **traditional programming techniques** would need to be updated to flag **“For U”** emails.

If spammers keep working around your spam filter, you will need to keep **writing new rules forever**.

In contrast, a **spam filter based on Machine Learning techniques** automatically notices that **“For U”** has become unusually frequent in spam flagged by users, and it **starts flagging them without your intervention**

Another area where Machine Learning shines is for **problems that either are too complex for traditional approaches or have no known algorithm.**

Eg 2 : Speech recognition.

Say you want to start simple and write a program **capable of distinguishing the words “one” and “two.”**

You might **notice that the word “two” starts with a high-pitch sound (“T”),** so you could **hardcode an algorithm that measures high-pitch sound intensity** and **use that to distinguish ones and twos** —but obviously this technique will **not scale to thousands of words spoken by millions of very different people in noisy environments and in dozens of languages.**

The best solution (at least today) is to **write an algorithm that learns by itself,** **given many example recordings for each word**

Finally, **Machine Learning can help humans learn**.

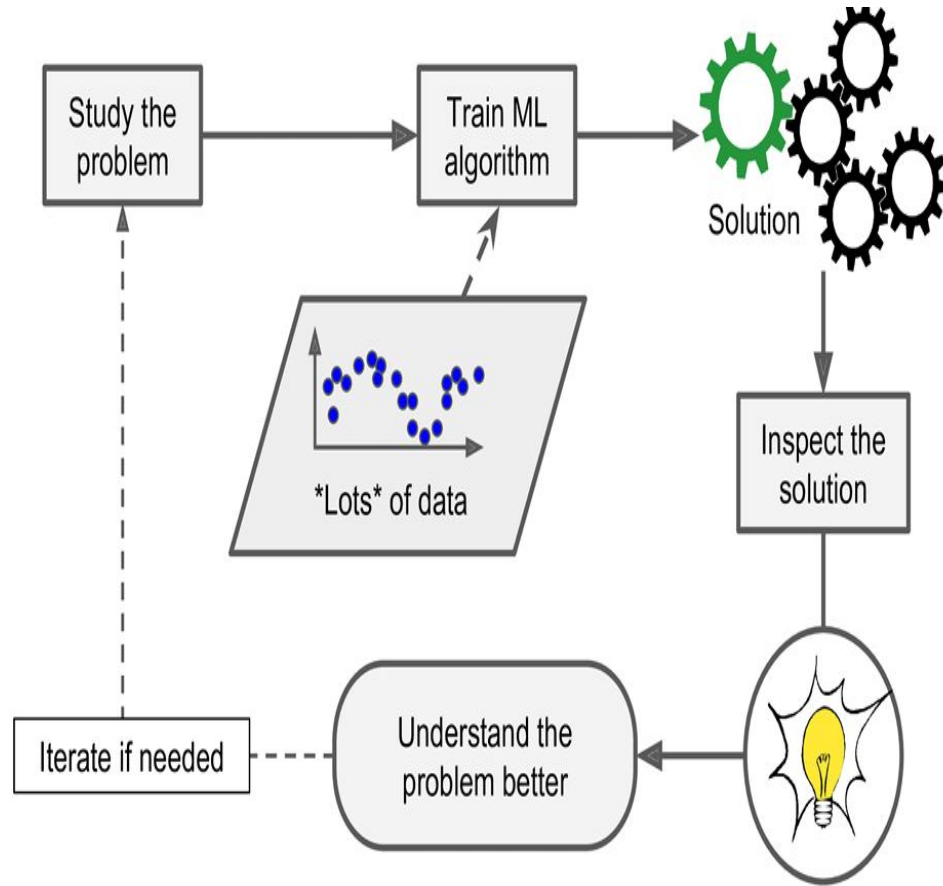


Figure 1-4. Machine Learning can help humans learn

ML algorithms can be **inspected to see what they have learned**.

For instance, once a **spam filter** has been **trained on enough spam**, it can **easily be inspected to reveal** the list of words and combinations of words that it **believes are the best predictors of spam**.

Sometimes **this will reveal unsuspected correlations or new trends**, and thereby lead to a **better understanding of the problem**.

Applying ML techniques to **dig into large amounts of data can help discover patterns that were not immediately apparent**.

This is called data mining

To summarize, Machine Learning is great for:

- **Problems for which existing solutions require a lot of fine-tuning or long lists of rules:** one Machine Learning algorithm can often **simplify code and perform better than the traditional approach.**
- **Complex problems for which using a traditional approach yields no good solution:** the best Machine Learning techniques can **perhaps find a solution.**
- **Fluctuating environments:** a Machine Learning system can **adapt to new data.**
- **Getting insights about complex problems** and large amounts of data

Examples of Applications

1. **Analyzing images of products on a production line to automatically classify them**

This is **image classification**, typically performed using convolutional neural networks

2. **Detecting tumors in brain scans**

This is **semantic segmentation**, where **each pixel in the image is classified** (as we want **to determine the exact location and shape of tumors**), typically using CNNs as well.

3. **Automatically classifying news articles**

This is **natural language processing** (NLP), and more specifically **text classification**, which can be tackled using recurrent neural networks (RNNs), CNNs, or Transformers

4. **Automatically flagging offensive comments on discussion forums**

This is also text classification, using the same **NLP tools**.

5. **Summarizing long documents automatically**

This is a **branch of NLP called text summarization**, again using the same tools.

6. **Creating a chatbot or a personal assistant**

This involves **many NLP components**, including natural language understanding (NLU) and question-answering modules.

7. **Forecasting your company's revenue next year, based on many performance metrics**

This is a regression task (i.e., predicting values) that may be tackled using any regression model

8. Making your app react to voice commands

This is **speech recognition**, which requires **processing audio samples**: since they are long and complex sequences, they are typically processed using RNNs, CNNs.

9. Detecting credit card fraud

This is **anomaly detection**

10. Segmenting clients based on their purchases so that you can design a different marketing strategy for each segment

This is **clustering**

11. Representing a complex, high-dimensional dataset in a clear and insightful diagram

This is **data visualization**, often involving **dimensionality reduction techniques**.

12. Recommending a product that a client may be interested in, based on past purchases

This is a **recommender system**.

One approach is to feed past purchases (and other information about the client) to an artificial neural network and get it to output the most likely next purchase. This neural net would typically be trained on past sequences of purchases across all clients

13. Building an intelligent bot for a game

This is often tackled using **Reinforcement Learning** - to **pick the actions that will maximize their rewards over time** (e.g., a bot may get a reward every time the player loses some life points), **within a given environment** (such as the game).

Eg: The famous AlphaGo program that beat the world champion at the game of Go was built using RL

Types of Machine Learning Systems

There are so many **different types of Machine Learning systems** that it is useful to classify them in broad categories, based on the following criteria:

1. **Whether or not they are trained with human supervision**
(supervised, unsupervised, semisupervised, and Reinforcement Learning)
2. **Whether or not they can learn incrementally on the fly** (online versus batch learning)
3. **Whether they work by simply comparing new data points to known data points, or instead by detecting patterns in the training data and building a predictive model**, much like scientists do
(instance-based versus model-based learning)

Supervised/Unsupervised Learning

Machine Learning systems can be **classified according to the amount and type of supervision they get during training.**

There are four major categories:

- supervised learning,
- unsupervised learning,
- Semisupervised learning, and
- Reinforcement Learning.

Supervised learning

In supervised learning, the **training set** you feed to the algorithm includes the **desired solutions, called labels**

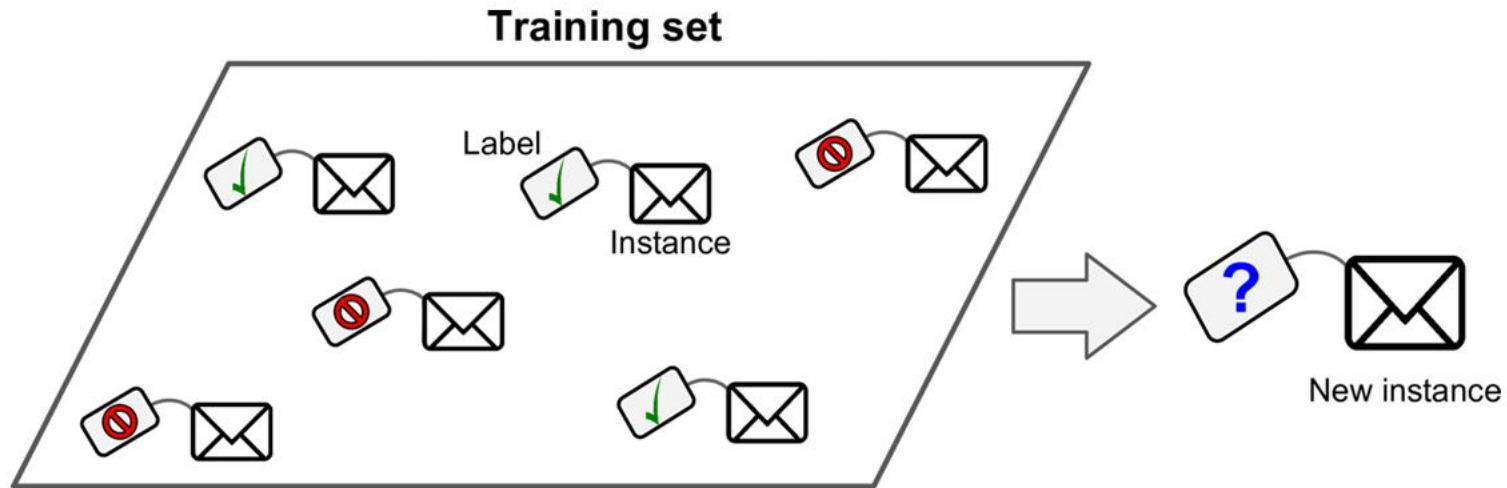


Figure 1-5. A labeled training set for spam classification (an example of supervised learning)

1. A typical supervised learning task is classification.

Example: Spam filter : it is **trained with many example emails along with their class (spam or ham)**, and it must learn how to classify new emails.

2. Another typical task is to **predict a target numeric value, such as the price of a car, given a set of features** (mileage, age, brand, etc.) called **predictors**. This sort of task is called regression .

To **train the system**, you need to **give it many examples of cars, including both their predictors and their labels** (i.e., their prices)

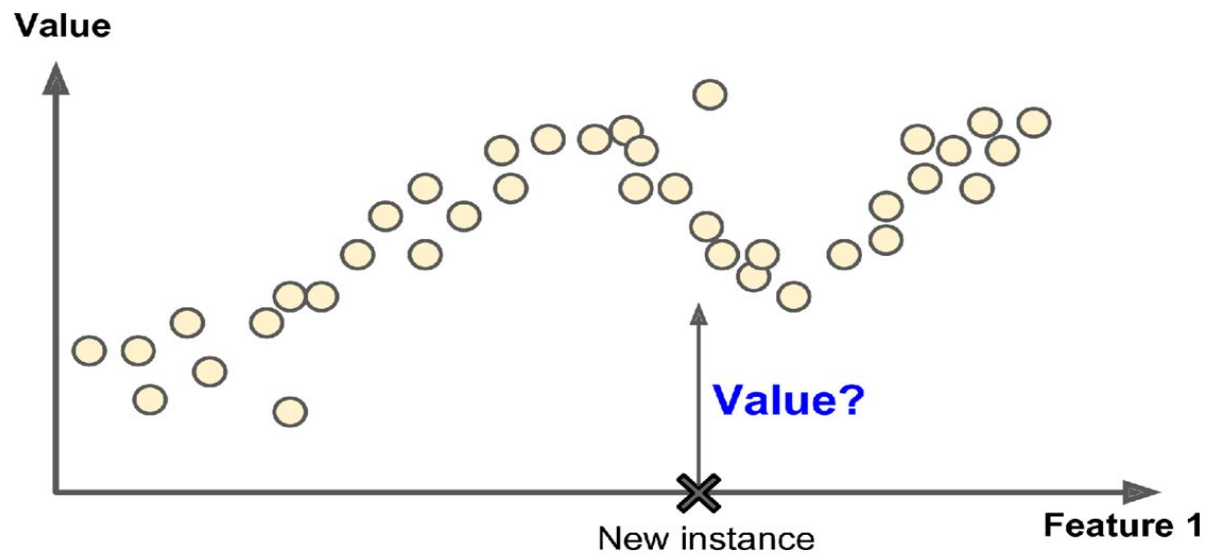


Figure 1-6. A regression problem: predict a value, given an input feature (there are usually multiple input features, and sometimes multiple output values)

Here are some of the most important supervised learning algorithms:

k-Nearest Neighbors

Linear Regression

Logistic Regression

Support Vector Machines (SVMs)

Decision Trees and Random Forests

Neural networks

Unsupervised learning

In unsupervised learning, the **training data is unlabeled** . The system tries to learn without a teacher

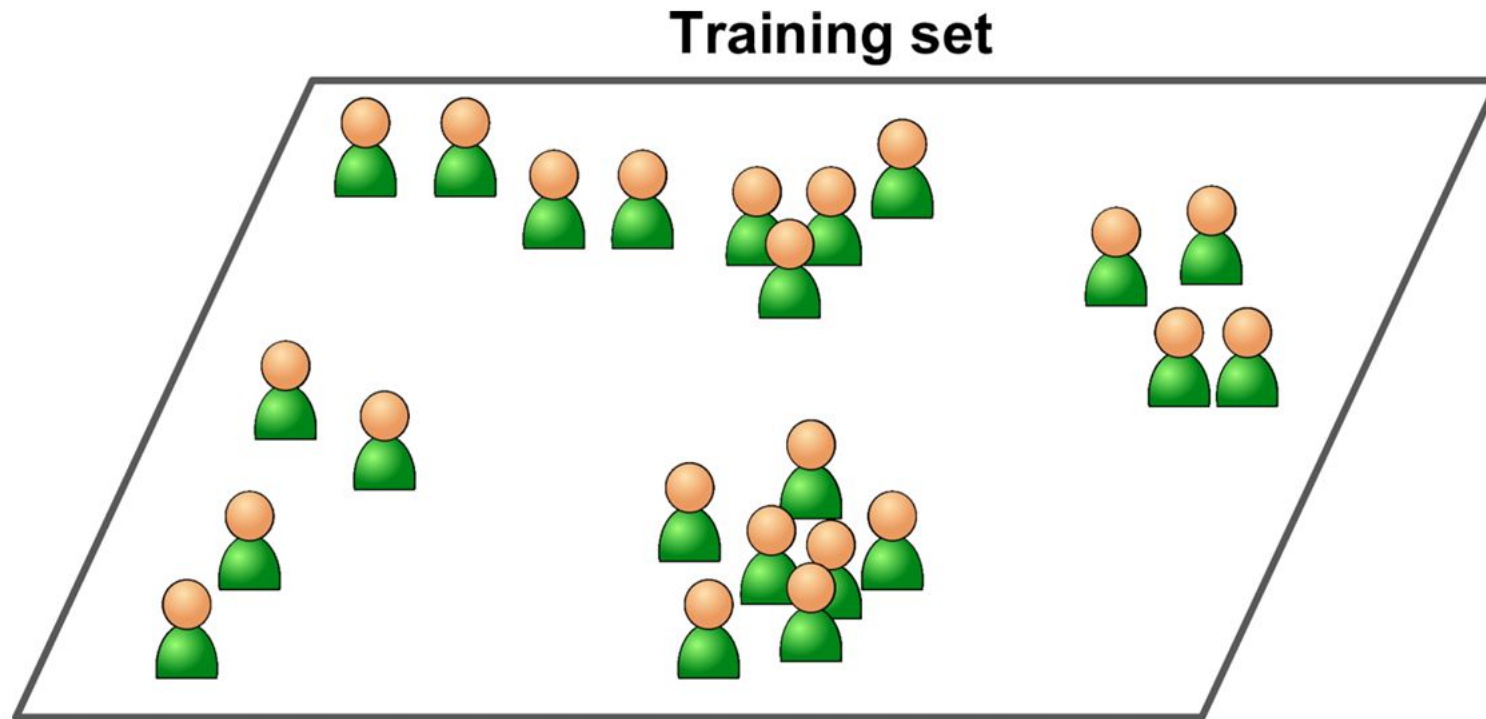


Figure 1-7. An unlabeled training set for unsupervised learning

Here are some of the most important unsupervised learning algorithms

Clustering

- K-Means
- DBSCAN
- Hierarchical Cluster Analysis (HCA)

Anomaly detection and novelty detection

- One-class SVM
- Isolation Forest

Visualization and dimensionality reduction

- Principal Component Analysis (PCA)
- Kernel PCA
- Locally Linear Embedding (LLE)
- t-Distributed Stochastic Neighbor Embedding (t-SNE)

Association rule learning

- Apriori
- Eclat

Example : 1. **Blog Visitors**

A lot of data about your blog's visitors.

You may want to **run a clustering algorithm to try to detect groups of similar visitors**.

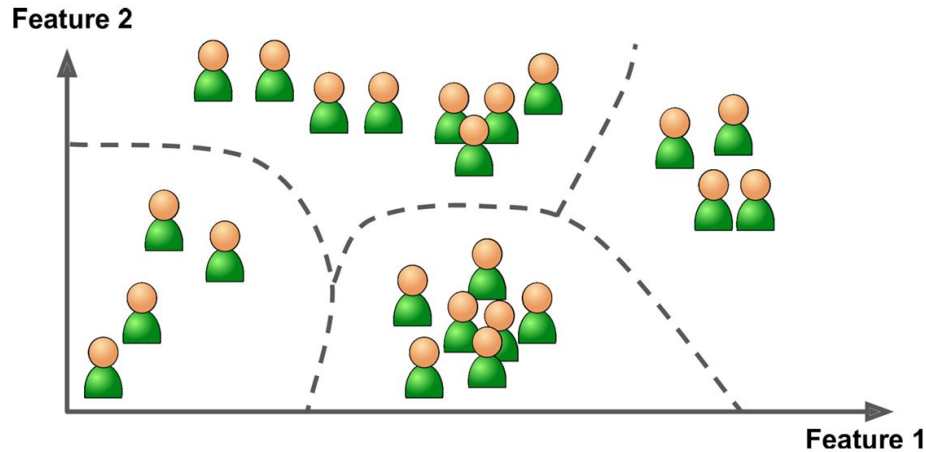


Figure 1-8. Clustering

At no point do you tell the algorithm which group a visitor belongs to: it finds those connections without your help.

It might **notice that 40% of your visitors are males who love comic books and generally read your blog in the evening, while 20% are young sci-fi lovers who visit during the weekends.**

If you use a hierarchical clustering algorithm, it may also **subdivide each group into smaller groups. This may help you target your posts for each group.**

Eg 2: Visualization algorithms are also good examples of unsupervised learning algorithms:

You feed them a lot of complex and unlabeled data, and they output a 2D or 3D representation of your data that can easily be plotted

These algorithms try to preserve as much structure as they can (e.g., trying to keep separate clusters in the input space from overlapping in the visualization) so that you can understand how the data is organized and perhaps identify unsuspected patterns

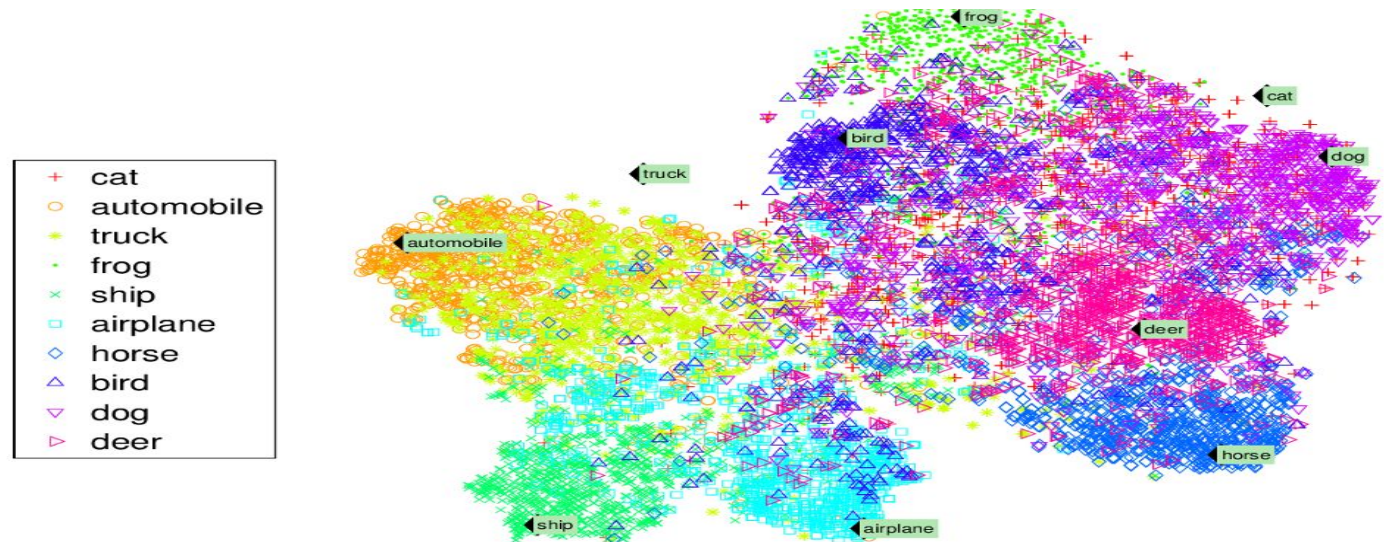


Figure 1-9. Example of a t-SNE visualization highlighting semantic clusters³

A related task is **dimensionality reduction**, in which the goal is to **simplify the data without losing too much information**.

One way to do this is to **merge several correlated features into one**.

Example:

A car's mileage + strongly correlated with its age

□ **dimensionality reduction** algorithm will merge them into one feature that represents the

□ **car's wear and tear**.

This is called **feature extraction**

Another important unsupervised task - **anomaly detection**

Examples

- **Detecting unusual credit card transactions to prevent fraud**
- **Catching manufacturing defects, or**
- **Automatically removing outliers from a dataset** before feeding it to another learning algorithm.

Method:

- The **system is shown mostly normal instances** during training,
- so it **learns to recognize them**
- then, **when it sees a new instance, it can tell whether it looks like a normal one or whether it is likely an anomaly**



Figure 1-10. Anomaly detection

A very similar task is **novelty detection**:

It aims to detect **new instances that look different from all instances in the training set.**

Example:

Thousands of pictures of dogs, and **1% of these pictures represent Chihuahuas**, then a novelty detection algorithm **should not treat new pictures of Chihuahuas as novelties.**

On the other hand, **anomaly detection algorithms may consider these dogs as so rare and so different from other dogs that they would likely classify them as anomalies**



Finally, another common unsupervised task **is association rule learning,**

Goal- To dig into large amounts of data and discover interesting relations between attributes.

Example- Supermarket

Running an association rule on your sales logs may reveal that people who purchase barbecue sauce and potato chips also tend to buy steak.

Thus, you may want to place these items close to one another

Semisupervised learning

Since **labeling data is usually time-consuming and costly**, there are often **plenty of unlabeled instances, and few labeled instances.**

Some **algorithms can deal with data that's partially labeled.**

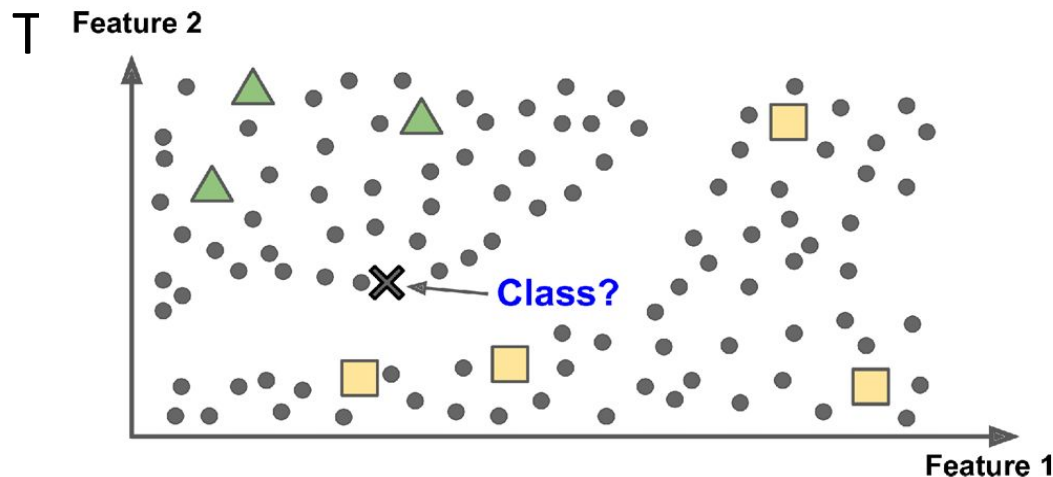


Figure 1-11. Semisupervised learning with two classes (triangles and squares): the unlabeled examples (circles) help classify a new instance (the cross) into the triangle class rather than the square class, even though it is closer to the labeled squares

Example: Photo-hosting services, such as **Google Photos**

Unsupervised learning:

Automatically recognizes that the **same person A shows up in photos 1, 5, and 11, while another person B shows up in photos 2, 5, and 7.**

Now all the **system needs is for you to tell it who these people are.**

Just add one label per person and it is able to name everyone in every photo, which is useful for searching photos

Reinforcement Learning

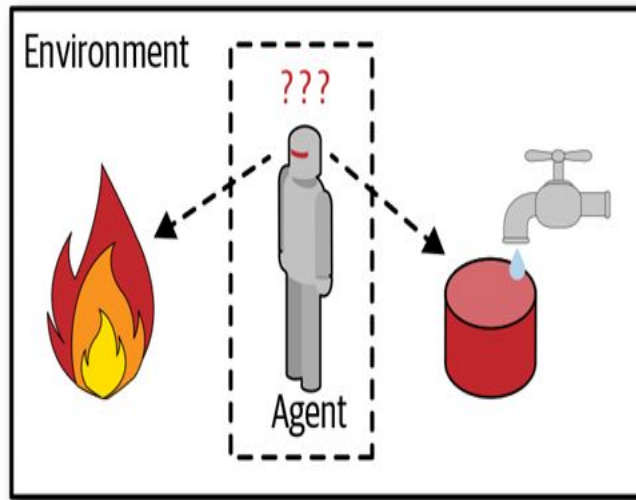
Reinforcement Learning is a very **different beast**.

The **learning system**, called an **agent** in this context, can

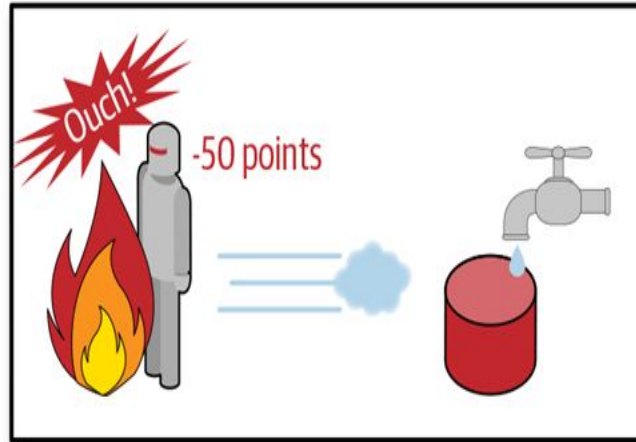
- **observe the environment,**
- **select and perform actions, and**
- **get rewards/penalty in return.**

It must **then learn by itself** what is the **best strategy**, called a **policy**, to get the **most reward over time**.

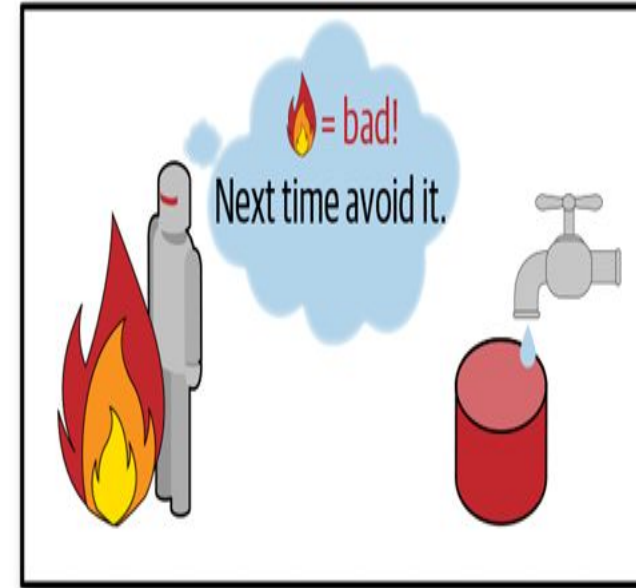
A **policy** defines what action the agent should choose when it is in a given **situation**



- 1 Observe
- 2 Select action using policy



- 3 Action!
- 4 Get reward or penalty



- 5 Update policy (learning step)
- 6 Iterate until an optimal policy is found

Figure 1-12. Reinforcement Learning

Batch and Online Learning

Another criterion used to classify Machine Learning systems is **whether or not the system can learn incrementally from a stream of incoming data**

Batch learning

In batch learning, the **system is incapable of learning incrementally:**
it must be **trained using all the available data.**

This will **generally take a lot of time and computing resources, so it is typically done offline.**

First the system is trained, and then it is launched into production and runs without learning anymore; it just applies what it has learned.

This is called offline learning.

If you want a batch learning system to know about new data you need to train a new version of the system from scratch on the full dataset (not just the new data, but also the old data), then stop the old system and replace it with the new one

Drawbacks:

- **Training** using the **full set of data can take many hours**, so you would typically train a new system only every **24 hours or even just weekly**.
- Also, **training on the full set of data requires a lot of computing resources**
- (CPU, memory space, disk space, disk I/O, network I/O, etc.).
- If you have **a lot of data and you automate your system to train from scratch every day, it will end up costing you a lot of money**.
- If the **amount of data is huge**, it may even **be impossible to use a batch learning algorithm**.
- Finally, if **your system needs to be able to learn autonomously and it has limited resources** then carrying around large amounts of training data and taking up a **lot of resources to train for hours every day is a showstopper**

Online learning

In online learning, **train** the

- **system incrementally by feeding it data instances sequentially, either individually or in small groups called mini batches.**
- **Each learning step is fast and cheap, so the system can learn about new data on the fly, as it arrives**

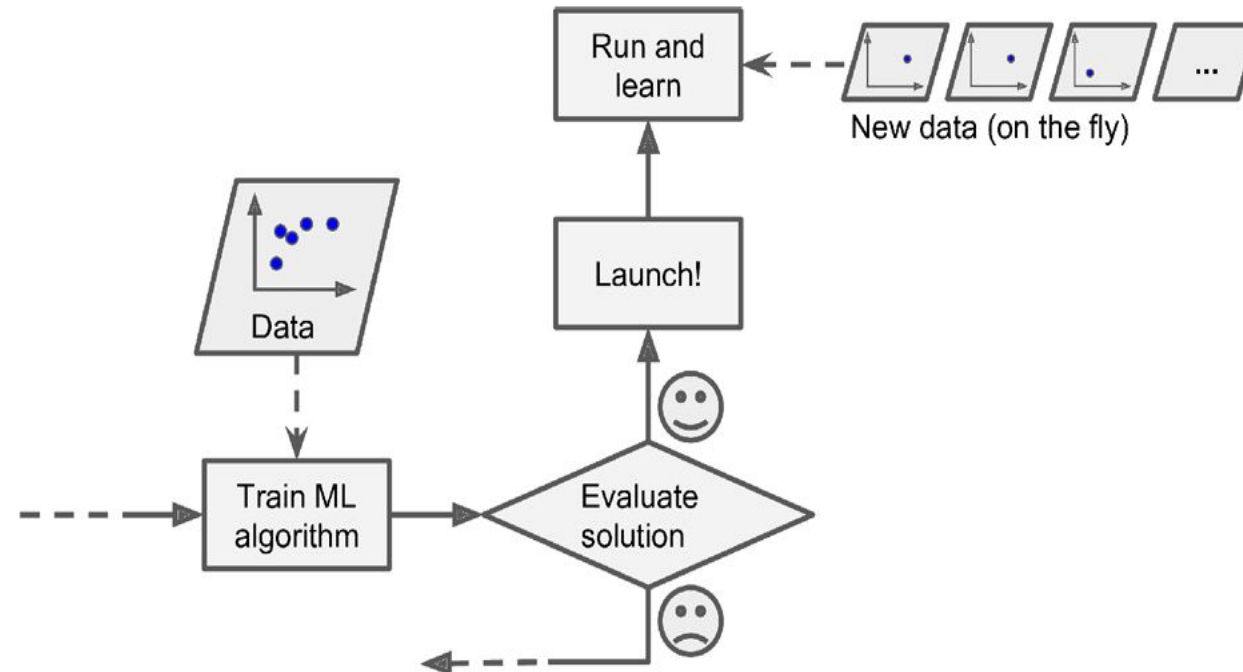


Figure 1-13. In online learning, a model is trained and launched into production, and then it keeps learning as new data comes in

Advantages:

- Online learning is **great for systems that receive data as a continuous flow** (e.g., stock prices) and need to **adapt to change rapidly or autonomously**.
- It is also a **good option if you have limited computing resources**:
- Once an **online learning system has learned about new data instances**, it does not need them anymore, so you can **discard them**
- This can save a huge amount of space

Online learning algorithms can also be used to **train systems on huge datasets that cannot fit in one machine's main memory - out of-core learning.**

The algorithm
loads part of the data,
runs a training step on that data, and
repeats the process until it has run on all of the data

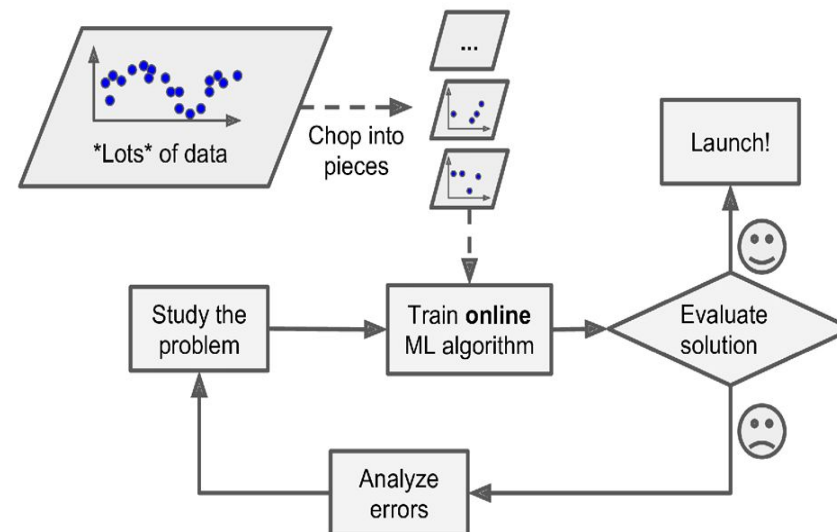


Figure 1-14. Using online learning to handle huge datasets

Learning Rate: One important parameter of online learning systems is **how fast they should adapt to changing data:**

If you set a **high learning rate**, then **your system will rapidly adapt to new data**, but it **will also tend to quickly forget the old data**

Conversely, if you set a **low learning rate**, the **system will have more inertia;**
that is, it will learn more slowly, but it will also be **less sensitive to noise in the new data or to sequences of nonrepresentative data points (outliers)**

Instance-Based Versus Model-Based Learning

One more way to **categorize Machine Learning systems** is by how they **generalize**.

Most Machine Learning tasks are about making predictions.

This means that **given a number of training examples**, the **system needs to be able to generalize to examples it has never seen before**.

There are **two main approaches to generalization**:
instance-based learning and
model-based learning.

Instance-based learning

i) Trivial form of learning is simply to **learn by heart**.

If you were to **create a spam filter this way**, it would **just flag all emails that are identical to emails that have already been flagged by users**

ii) Instead of just flagging emails that are identical to known spam emails, **your spam filter could be programmed to also flag emails that are very similar to known spam emails**.

This requires a **measure of similarity between two emails**.

A (very basic) **similarity measure between two emails could be to count the number of words they have in common**.

The system would **flag an email as spam** if it has many words in common with a **known spam email**.

This is **called instance-based learning**: the system learns the examples by heart, then **generalizes to new cases by comparing them to the learned examples using a similarity measure**.

The new instance would be classified as a triangle because the majority of the most similar instances belong to that class

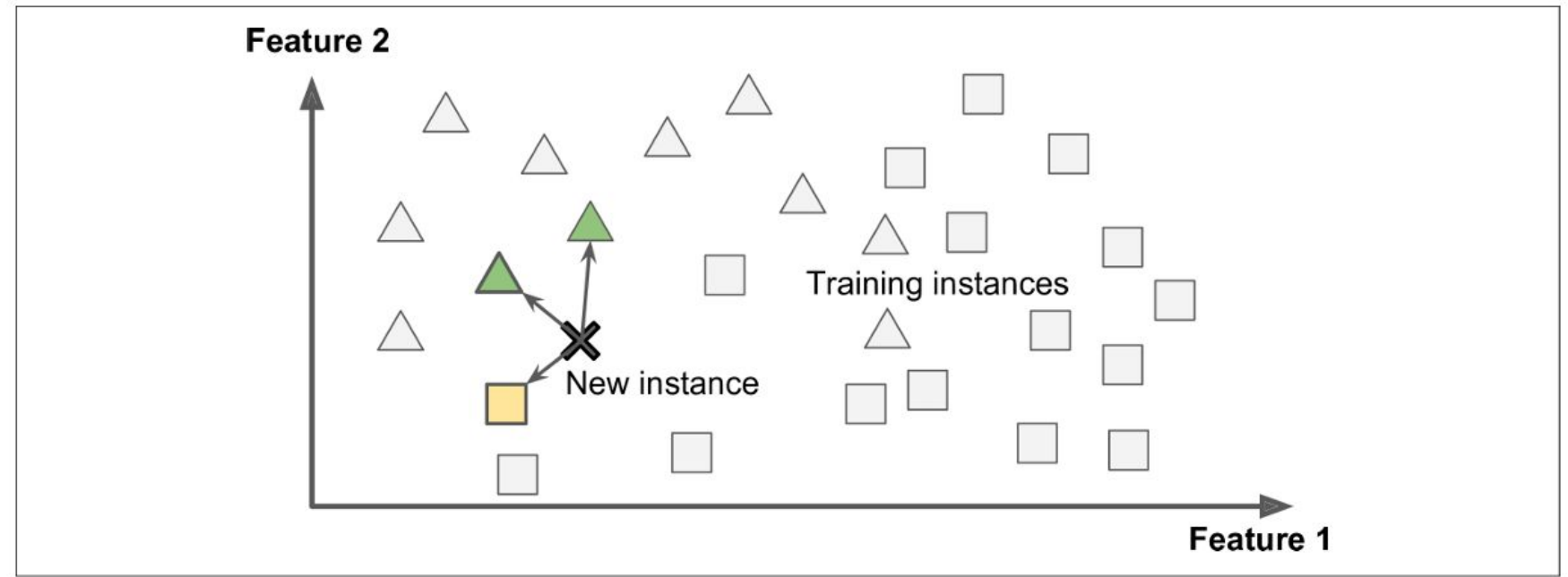


Figure 1-15. Instance-based learning

Model-based learning

Another way to generalize from a set of examples is to build a model of these examples, then use that model to make predictions.

This is called model-based learning

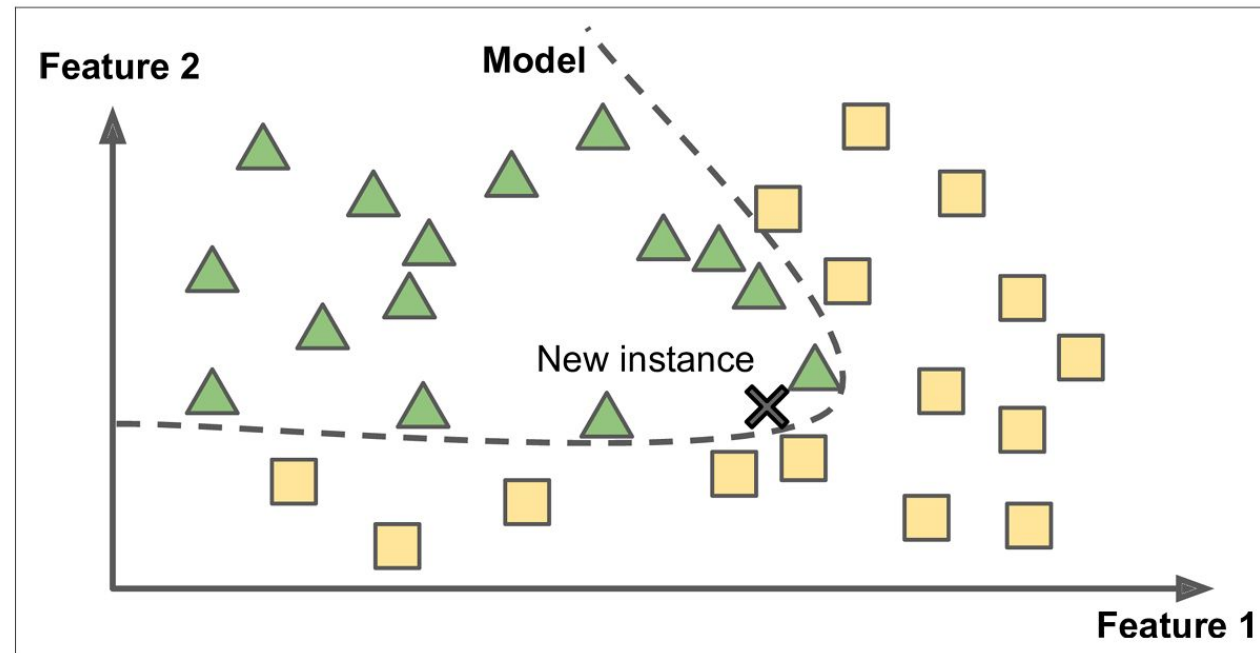


Figure 1-16. Model-based learning

Example:

Suppose you want to know if money makes people happy ?

Download the **Better Life Index** data from the OECD's website as well as stats about **GDP per capita** from the IMF's website.

Then you **join the tables and sort by GDP per capita**.

Table shows an excerpt of what you get.

Table 1-1. Does money make people happier?

Country	GDP per capita (USD)	Life satisfaction
Hungary	12,240	4.9
Korea	27,195	5.8
France	37,675	6.5
Australia	50,962	7.3
United States	55,805	7.2

Let's plot the data for a few random countries (**Figure 1-17**).

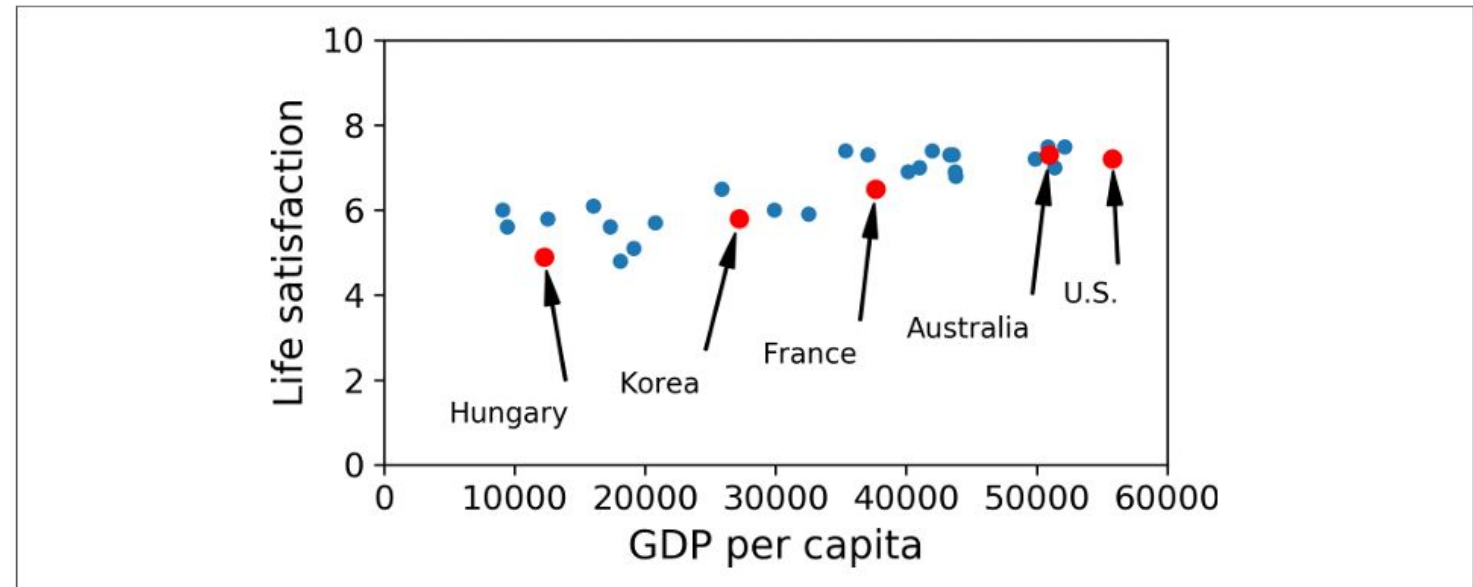


Figure 1-17. Do you see a trend here?

There does **seem to be a trend here!**

Life satisfaction goes up more or less linearly as the country's GDP per capita increases.

So you **decide to model life satisfaction as a linear function of GDP per capita.**

This step is called model selection:

you selected a linear model of life satisfaction with just one attribute, GDP per capita (Equation 1-1).

Equation 1-1. A simple linear model

$$\text{life_satisfaction} = \theta_0 + \theta_1 \times \text{GDP_per_capita}$$

This model has two model parameters, θ_0 and θ_1 .

By tweaking these parameters, you can make your model represent any linear function, as shown

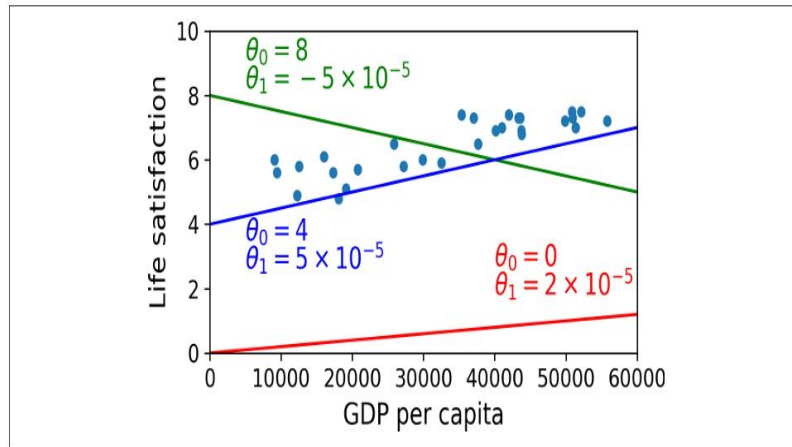


Figure 1-18. A few possible linear models

Parameter values are $\theta_0 = 4.85$ and $\theta_1 = 4.91 \times 10^{-5}$.

Now the model fits the training data as closely as possible (for a linear model), as you can see in Figure 1-19.

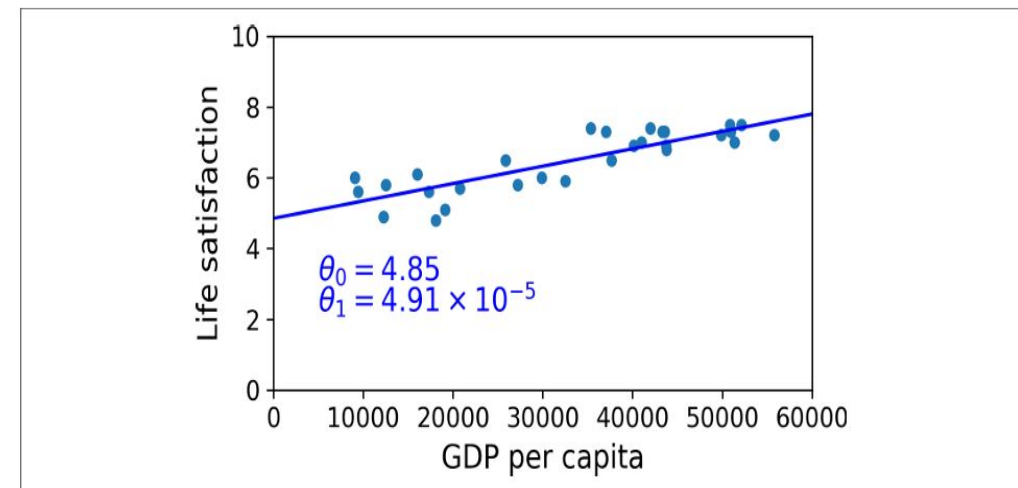


Figure 1-19. The linear model that fits the training data best

You are finally ready to run the model to make predictions.

For example, say you want to know **how happy Cypriots** are, and the OECD data does not have the answer.

Fortunately, you can **use your model to make a good prediction**: you look up Cyprus's GDP per capita, find \$22,587, and

then apply your model and find that life satisfaction is likely to be somewhere around $4.85 + 22,587 \times 4.91 \times 10^{-5} = 5.96$.

Main Challenges of Machine Learning

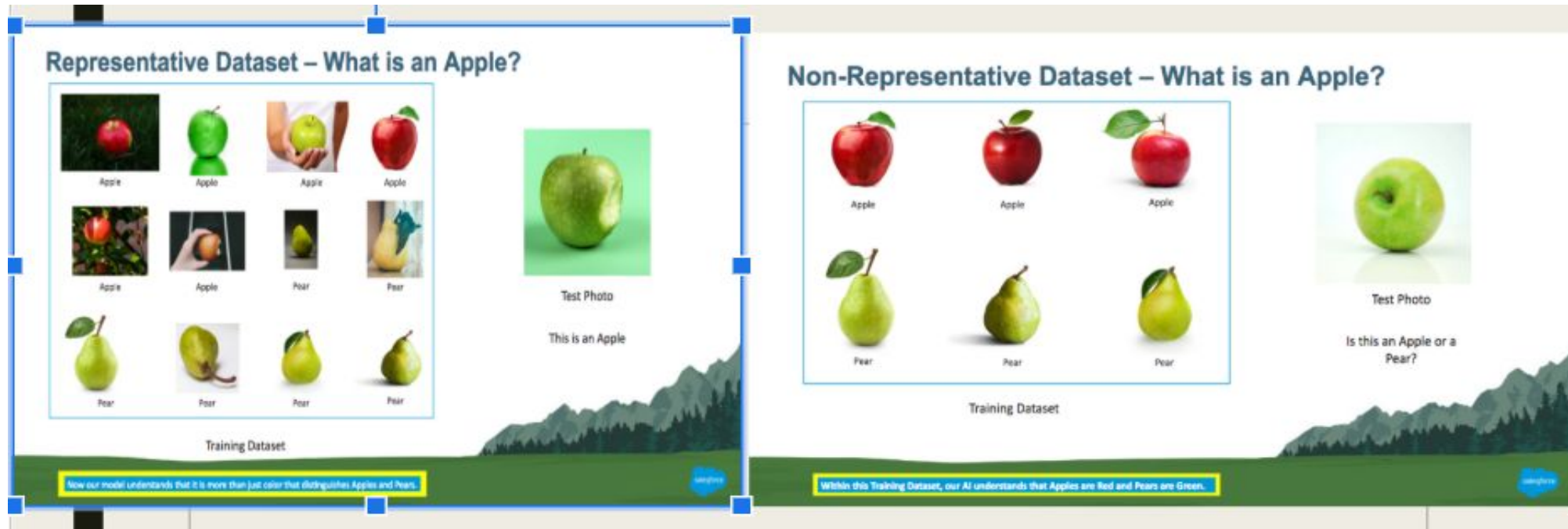
The main task is to **select a learning algorithm and train it on some data**, the two things that can go wrong are “**bad algorithm**” and “**bad data.**”

1. Let's start with examples of bad data

a. Insufficient Quantity of Training Data

For a toddler to learn what an apple is, --all it takes is for you to point to an apple and say “apple”.

Now the child is able to recognize apples in all sorts of colors and shapes.



Machine Learning in turns takes a lot of data for most Machine Learning algorithms to work properly.

Even for very simple problems typically need thousands of examples, and for complex problems such as image or speech recognition need millions of examples– Eg Cat: need various cats with different features

Nonrepresentative Training Data

In order to generalize well, it is crucial that your training data be representative of the new cases you want to generalize to.

Example 1: If **model is trained only for few types of dogs** and if **test data has a dog which is not been trained** then the **model might not make** accurate predictions.

Example 2: The **set of countries** we used earlier for **training the linear model** was **not perfectly representative**; a **few countries were missing**. Figure shows what the data looks like when you add the missing countries

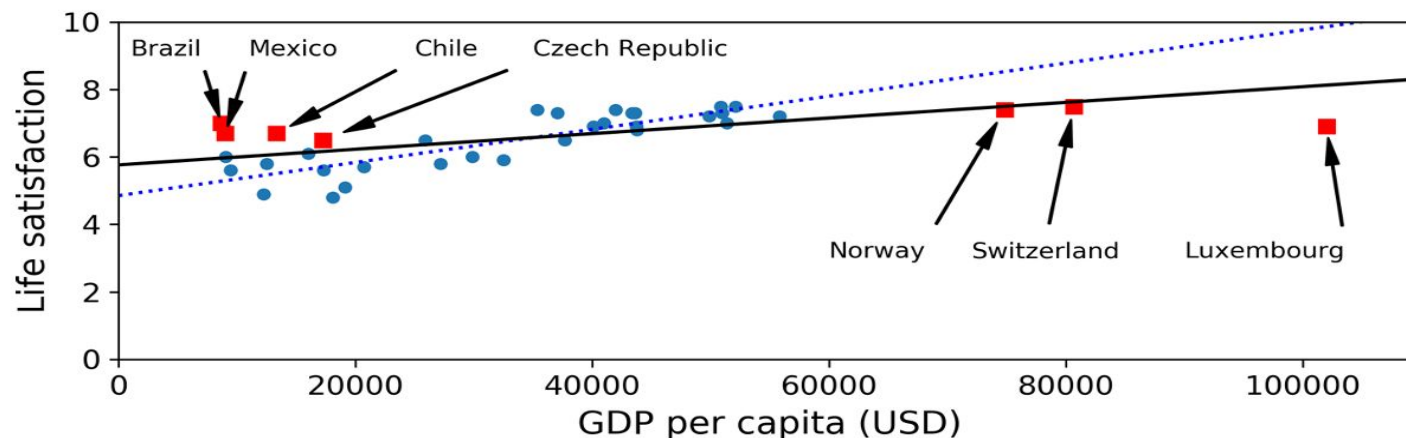


Figure 1-21. A more representative training sample

If you train a linear model on this data, you get the solid line, while the old model is represented by the dotted line. As you can see, not only does adding a few missing countries significantly alter the model, but it makes it clear that such a simple linear model is probably never going to work well.

Poor-Quality Data

If your training data is full of errors, outliers, and noise , it will make it harder for the system to detect the underlying patterns, so your system is less likely to perform well.

It is often well worth the effort to spend time cleaning up your training data.

The following are a couple of examples of when you'd want to clean up training data:

If some instances are clearly outliers, it may help to simply discard them or try to fix the errors manually.

If some instances are missing a few features (e.g., 5% of your customers did not specify their age), you must decide

- whether you want to ignore this attribute altogether,
- ignore these instances,
- fill in the missing values (e.g., with the median age),
- or train one model with the feature and one model without it.

Irrelevant Features

The system will only be capable of learning if the training data contains enough relevant features and not too many irrelevant ones.

A critical part of the success of a Machine Learning project is coming up with a good set of features to train on.

This process, called **feature engineering**, involves the following steps:

- **Feature selection** (selecting the most useful features to train on among existing features)
- **Feature extraction** (combining existing features to produce a more useful one —dimensionality reduction algorithms can help)
- **Creating new features by gathering new data**

2. A couple of examples of bad algorithms

Overfitting the Training Data

Example: Tempted to say that all taxi drivers in that country are thieves.

–Overgeneralizing

In Machine Learning this is called **overfitting**--the **model performs well on the training data, but it does not generalize well.**

$X=2y$ $x=2,3,4$ $y=4,6,8$ --by heart ---training is good If test=10 then $y=25$ --X

Complex models such as deep neural networks can detect subtle patterns in the data, but if the training set is noisy, or if it is too small (which introduces sampling noise), then the model is likely to detect patterns in the noise itself.

Obviously these patterns will not generalize to new instances.

For example, say you **feed your life satisfaction model many more attributes, such as the country's name.**

In that case, a **complex model may detect patterns like the fact that all countries in the training data with a w in their name have a life satisfaction greater than 7: New Zealand (7.3), Norway (7.4), Sweden (7.2), and Switzerland (7.5).**

How confident are you that the **w-satisfaction rule generalizes to Rwanda or Zimbabwe?**

Overfitting happens when the **model is too complex relative to the amount and noisiness of the training data.**

Here are possible solutions:

Simplify the model by selecting one with fewer parameters (e.g., a linear model rather than a high-degree polynomial model),

by reducing the number of attributes in the training data, or by constraining the model.

Gather more training data

Reduce the noise in the training data (e.g., fix data errors and remove outliers).

Underfitting the Training Data

Underfitting is the **opposite of overfitting**:

It occurs when your **model is too simple to learn the underlying structure of the data**.

For example, a **linear model of life satisfaction is prone to underfit**;

reality is just more complex than the model, so its predictions are bound to be inaccurate, even on the training examples.

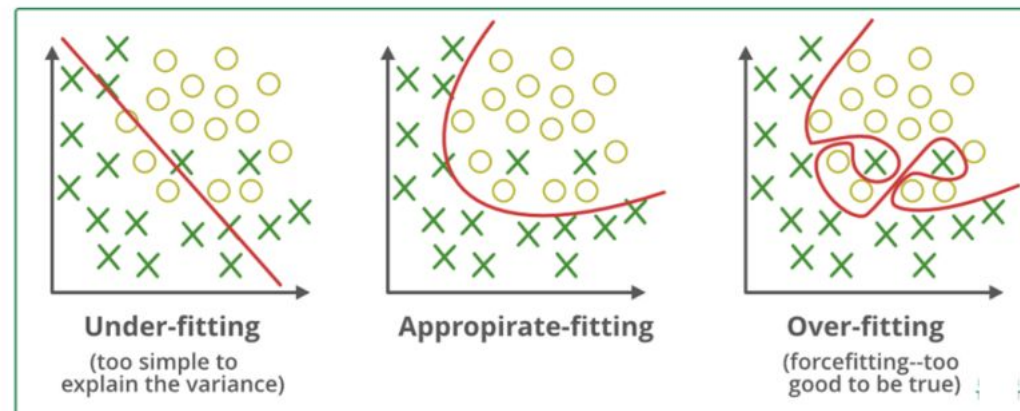
Example: $y=2x$ may learn as $y=x$ due to lack of training data : **Not able to learn the complete pattern of underlying data**

Here are the main options for fixing this problem:

- Select a **more powerful model, with more parameters.**
- Feed **better features to the learning algorithm** (feature engineering).
- **Reduce the constraints** on the model

Overfitting the Training Data: “Overfitting the training data” means a model has learned the training data too closely, capturing even minute details and noise, resulting in poor performance on new, unseen data, essentially "memorizing" the training set instead of learning generalizable patterns.

“Underfitting the training data” means that a model is not learning enough from the training data, failing to capture the underlying patterns and trends, resulting in poor performance on both the training set and new, unseen data due to its inability to generalize effectively;



Training, Testing and Validation Data

Training Data: This is the **primary dataset** used to train the machine learning model, where the algorithm learns patterns and relationships from the labeled data to make predictions.

Validation Data: A separate portion of the data used to monitor the model's performance during training, allowing for adjustments to hyperparameters and **preventing overfitting**.

Testing Data: Once the model is fully trained, the **testing data is used to evaluate its final accuracy and generalization ability on completely unseen data**

Data Splitting: To effectively utilize these sets, **the original dataset is usually split into training, validation, and testing sets, often following a standard 80/10/10 ratio (80% training, 10% validation, 10% testing).**

End-to-End Machine Learning Project

End-to-End Machine Learning Project

Main Steps:

1. Frame the problem
 2. Get the data.
 3. Discover and visualize the data to gain insights.
 4. Prepare the data for Machine Learning algorithms.
 5. Select a model and train it.
 6. Fine-tune your model.
 7. Present your solution.
 8. Launch, monitor, and maintain your system.
-

California Housing Prices dataset

- ❑ Source: github.com/ageron/data/raw/amin/housing.tgz
- ❑ Purpose: It is **commonly used for regression tasks, such as predicting the median house value in a district based on various features.**
- ❑ Size: The dataset contains approximately **20,640 rows (districts) and 10 columns (features).**

Sample screenshot of "housing.csv" , one row per district



longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	ocean_proximity	median_house_value
-122.23	37.88	41	880	129	322	126	8.3252	NEAR BAY	452600
-122.22	37.86	21	7099	1106	2401	1138	8.3014	NEAR BAY	358500
-122.24	37.85	52	1467	190	496	177	7.2574	NEAR BAY	352100
-122.25	37.85	52	1274	235	558	219	5.6431	NEAR BAY	341300
-122.25	37.85	52	1627	280	565	259	3.8462	NEAR BAY	342200
-122.25	37.85	52	919	213	413	193	4.0368	NEAR BAY	269700

California Housing Prices dataset

Column Name	Description	Data Type	Relevance
longitude	Longitude of the housing district	Numeric	Geographic location feature. Useful for spatial analysis.
latitude	Latitude of the housing district	Numeric	Geographic location feature. Useful for spatial analysis.
housing_median_age	Median age of houses in the district	Numeric	Indicates the age of housing stock. Older houses may have lower values.
total_rooms	Total number of rooms in the district	Numeric	Reflects the size of housing in the district.
total_bedrooms	Total number of bedrooms in the district	Numeric	Indicates the capacity of housing in the district.
population	Total population in the district	Numeric	Reflects the density of the district.
households	Total number of households in the district	Numeric	Indicates the number of families or housing units.
median_income	Median income of households in the district (in tens of thousands of dollars)	Numeric	Strong predictor of house value. Higher income often correlates with higher prices.
median_house_value	Median house value in the district (target variable)	Numeric	The target variable for regression tasks.
ocean_proximity	Proximity to the ocean (categorical: <1H OCEAN, INLAND, NEAR OCEAN, NEAR BAY, ISLAND)	Categorical	Indicates the location's proximity to the ocean, which can affect house value.

California Housing Prices dataset

1. Geographic Features:

- longitude and latitude can be used to create spatial visualizations or calculate distances to key locations.
- ocean_proximity is a categorical feature that can be encoded for use in models.

2. Housing Characteristics:

- housing_median_age, total_rooms, total_bedrooms, and households describe the housing stock in the district.
- These features can be used to derive new features, such as:
 - i. **Rooms per household:** $\text{total_rooms} / \text{households}$
 - ii. **Bedrooms per room:** $\text{total_bedrooms} / \text{total_rooms}$
 - iii. **Population per household:** $\text{population} / \text{households}$

3. Economic Indicator:

- median_income is a strong predictor of median_house_value and is often used as a key feature in regression models.

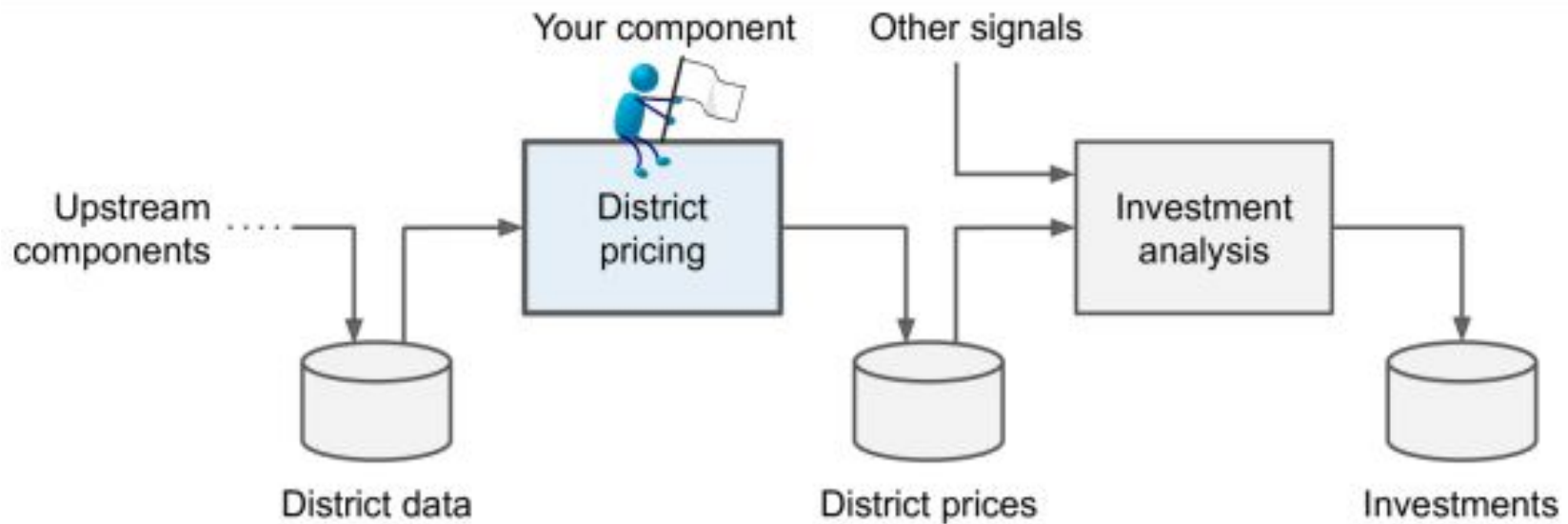
4. Target Variable:

- median_house_value is the target variable for regression tasks. **The goal is to predict this value based on the other features.**
-

A Machine Learning pipeline for real estate investments

A **sequence of data processing components** is called a data pipeline.

Pipelines are very **common in Machine Learning systems**, since there is a lot of data to manipulate and many data transformations to apply.



Dataframe

A DataFrame is a **2-dimensional labeled data structure** with columns of potentially different types, similar to a spreadsheet or SQL table.

What is a DataFrame?

- A **DataFrame** is a table-like structure with rows and columns.
- Each column represents a **feature** (or variable), and each row represents an **observation** (or record).
- Columns can have different data types (e.g., numbers, strings, dates).

Key Features of a DataFrame

Rows: Represent individual records (e.g., a student's data).

Columns: Represent attributes or features (e.g., name, age, grade).

Index: A unique identifier for each row (like row numbers in a spreadsheet).

Dataframe

Creating a DataFrame

```
import pandas as pd
# Create a dictionary with student data
data = { "Name": ["Alice", "Bob", "Charlie", "David"],
"Age": [20, 21, 19, 22],
"Grade": ["A", "B", "C", "A"] }

# Convert the dictionary into a DataFrame
df = pd.DataFrame(data)

# Display the DataFrame
print(df)
```

Output

	Name	Age	Grade
0	Alice	20	A
1	Bob	21	B
2	Charlie	19	C
3	David	22	A

Dataframe

Accessing Data in a DataFrame

```
print(df["Name"]) # Access the "Name" column
```

Output

```
0    Alice
1     Bob
2  Charlie
3   David
```

```
print(df.iloc[1]) # Access the second row (index 1)
```

Output

```
Name    Bob
Age     21
Grade    B
```

Dataframe

```
print(df.loc[2, "Age"]) # Access the age of the third student (index 2)
```

Output

```
19
```

Adding a New Column

```
df["City"] = ["New York", "Los Angeles", "Chicago", "Houston"]  
print(df)
```

Output

	Name	Age	Grade	City
0	Alice	20	A	New York
1	Bob	21	B	Los Angeles
2	Charlie	19	C	Chicago
3	David	22	A	Houston

Dataframe

Adding a row to existing frame

Method 1: Using loc to Add a New Row

Add a new row using loc

`df.loc[3] = ["David", 22, "A"]` # Index 3 is the new row

Display the updated DataFrame `print(df)`

Output

	Name	Age	Grade
0	Alice	20	A
1	Bob	21	B
2	Charlie	19	C
3	David	22	A

Dataframe

Adding a new row

Method 2: Using pd.concat()

```
# Create a DataFrame for the new row
new_row = pd.DataFrame({"Name": ["Eva"], "Age": [23], "Grade": ["B"]})

# Add the new row using pd.concat
df = pd.concat([df, new_row], ignore_index=True)

# Display the updated DataFrame
print(df)
```

Output

	Name	Age	Grade
0	Alice	20	A
1	Bob	21	B
2	Charlie	19	C
3	David	22	A
4	Eva	23	B

Python Code

To read data from .csv file

```
import pandas as pd
housing = pd.read_csv("housing.csv")
housing.head()
```

Output

longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	ocean_proximity	median_house_value
-122.23	37.88	41	880	129	322	126	8.3252	NEAR BAY	452600
-122.22	37.86	21	7099	1106	2401	1138	8.3014	NEAR BAY	358500
-122.24	37.85	52	1467	190	496	177	7.2574	NEAR BAY	352100
-122.25	37.85	52	1274	235	558	219	5.6431	NEAR BAY	341300
-122.25	37.85	52	1627	280	565	259	3.8462	NEAR BAY	342200

The head() method returns a specified number of rows, string from the top.
The head() method returns the first 5 rows if a number is not specified.

Python Code

housing.info()

#	Column	Non-Null	Count	Dtype
---	-----	-----	-----	-----
0	longitude	20640	non-null	float64
1	latitude	20640	non-null	float64
2	housing_median_age	20640	non-null	float64
3	total_rooms	20640	non-null	float64
4	total_bedrooms	20433	non-null	float64
5	population	20640	non-null	float64
6	households	20640	non-null	float64
7	median_income	20640	non-null	float64
8	median_house_value	20640	non-null	float64
9	ocean_proximity	20640	non-null	object

Python Code

```
housing["ocean_proximity"].value_counts()
```

```
<1H OCEAN    9136  
INLAND        6551  
NEAR OCEAN    2658  
NEAR BAY      2290  
ISLAND         5
```

Python Code

The describe() method shows a summary of the numerical attributes

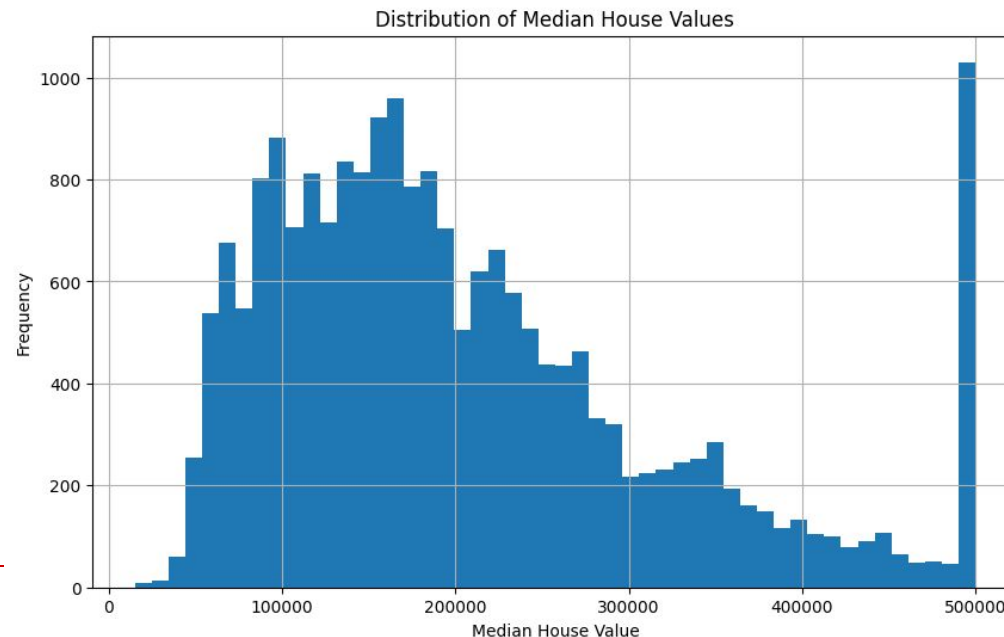
```
housing.describe()
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms
count	20640.000000	20640.000000	20640.000000	20640.000000	20433.000000
mean	-119.569704	35.631861	28.639486	2635.763081	537.870553
std	2.003532	2.135952	12.585558	2181.615252	421.385070
min	-124.350000	32.540000	1.000000	2.000000	1.000000
25%	-121.800000	33.930000	18.000000	1447.750000	296.000000
50%	-118.490000	34.260000	29.000000	2127.000000	435.000000
75%	-118.010000	37.710000	37.000000	3148.000000	647.000000
max	-114.310000	41.950000	52.000000	39320.000000	6445.000000

Python Code

A histogram shows the number of instances (on the vertical axis) that have a given value range (on the horizontal axis).

```
import matplotlib.pyplot as plt
# Plot a histogram of median house values
housing['median_house_value'].hist(bins=50, figsize=(10, 6))
plt.xlabel("Median House Value")
plt.ylabel("Frequency")
plt.title("Distribution of Median House Values")
plt.show()
```



Data snooping bias

- ❑ **Data snooping bias (also known as data dredging or p-hacking) is a common issue in machine learning and statistical analysis where the**
 - ❑ **model or analysis is overly tailored to the specific dataset used for training or testing, leading to**
 - ❑ **overly optimistic performance estimates and**
 - ❑ **poor generalization to new, unseen data.**
-

-
- Data snooping bias occurs when:
 - The **dataset is used multiple times to make decisions about the model** (e.g., feature selection, hyperparameter tuning, or model selection).
 - The **model is indirectly exposed to information from the test set during training, violating the principle of keeping the test set completely separate.**

 - This results in a model that performs well on the training or validation data but fails to generalize to new, unseen data.
-

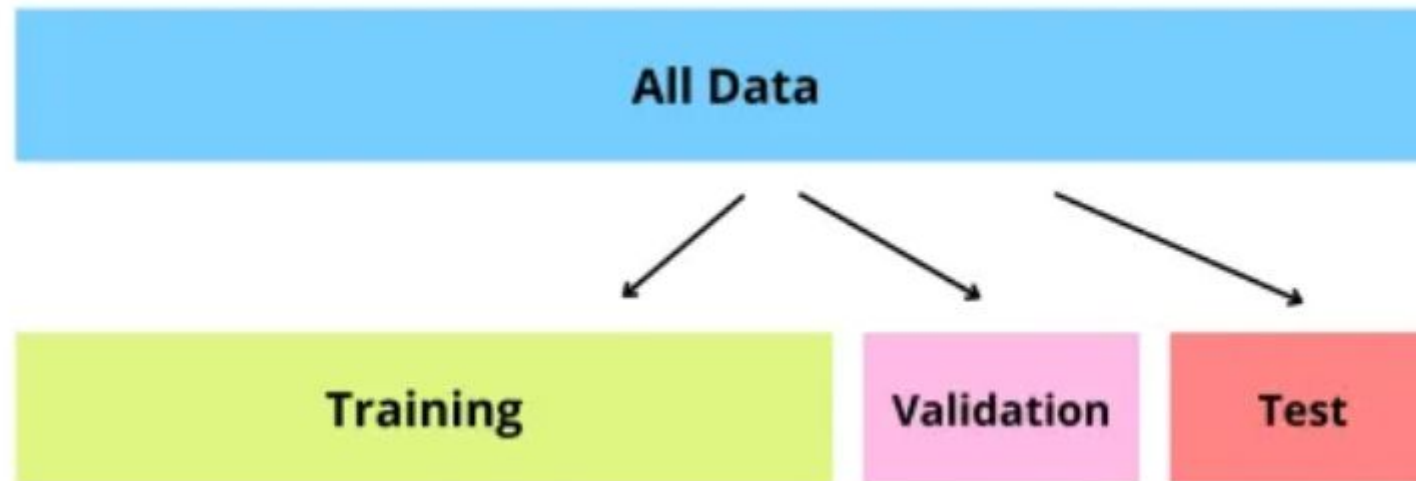
Train, Validation and Test Split of Data

Dividing a dataset into three distinct subsets:

Training set: Used to train the model

Validation set: Used to **fine-tune the model during training by evaluating its performance** on unseen data)

Test set: Used to **assess the final model's accuracy on completely new data**, providing an **unbiased evaluation** of its generalization ability.



Example

Let's assume we have a small dataset with 10 samples and 2 features (X) and a target variable (y):

```
import numpy as np
# Example dataset
X = np.array([[1, 2], [3, 4], [5, 6], [7, 8], [9, 10], [11, 12], [13, 14], [15, 16], [17, 18],
              [19, 20]])
y = np.array([0, 1, 0, 1, 0, 1, 0, 1, 0, 1])
```


Example

Step 1: Split into Train and Test Sets

First, we split the dataset into train and test sets using `train_test_split`. We'll use **80% of the data for training and 20% for testing**. **`random_state` parameter to ensure reproducibility.**

```
from sklearn.model_selection import train_test_split
# Split into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Output

```
Train set (X_train):
[[ 3  4]
 [19 20]
 [ 7  8]
 [17 18]
 [13 14]
 [ 5  6]
 [ 9 10]
 [15 16]]
Test set (X_test):
[[ 1  2]
 [11 12]]
```

Example

Step 2: Split Train Set into Train and Validation Sets

Next, we split the **train set** further into **train** and **validation** sets. We'll use 75% of the train set for training and 25% for validation.

```
# Split train set into train and validation sets
```

```
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train, test_size=0.25,  
random_state=42)
```

Output

```
Train set (X_train):  
[[19 20]  
 [17 18]  
 [ 5  6]  
 [15 16]  
 [ 7  8]  
 [13 14]]  
Validation set (X_val):  
[[ 3  4]  
 [ 9 10]]
```

Creating Test Set: Random and Stratified Sampling

Random Sampling

- **Definition:** Randomly splits the dataset into training and test sets without considering the distribution of the target variable or other features.
- **Advantage:** Simple and easy to implement.
- **Disadvantage:** May lead to biased test sets, especially if the dataset is imbalanced (e.g., one class is underrepresented).

```
import pandas as pd
from sklearn.model_selection import train_test_split
```

```
housing = pd.read_csv("housing.csv")
```

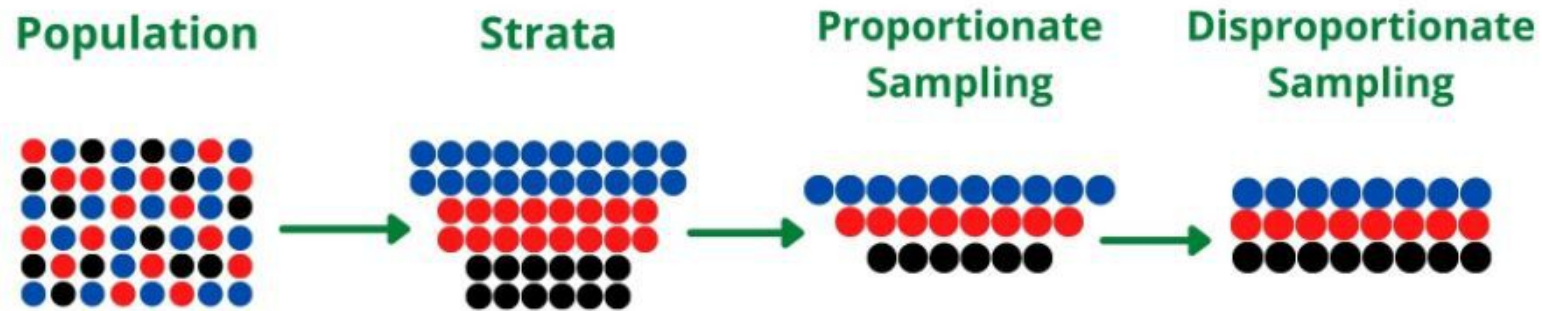
```
# Random sampling
```

```
train_set, test_set = train_test_split(housing, test_size=0.2, random_state=42)
```

Creating Test Set: Random and Stratified Sampling

Stratified Sampling

- **Definition:** Ensures that the **test set has the same distribution of a specific feature** (usually the target variable) **as the entire dataset**.
- **Advantage:** Reduces sampling bias and ensures that the **test set is representative of the overall dataset**.
- **Disadvantage:** Slightly more complex to implement.



Creating Test Set: Random and Stratified Sampling

When using `train_test_split` with the `stratify` parameter, the **splits will preserve the proportion of the target variable** (or any other specified feature) **in both the training and test sets**.

This is particularly useful for **imbalanced datasets** where one class is underrepresented.

```
# Split into train and test sets with stratification
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42, stratify=y)
```

For random sampling

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split

# Load the dataset
url =
"https://raw.githubusercontent.com/ageron/handson-ml2/master/datasets/housing/housing.csv"
housing = pd.read_csv(url)

# Separate into features (X) and target (y)
X = housing.drop("median_house_value", axis=1) # Features (all columns except the target)
y = housing["median_house_value"] # Target variable

# Split into train and test sets with stratification
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

-
- ❑ Median income is a very important attribute to predict median housing prices.
 - ❑ Ensure that the test set is representative of the various categories of incomes in the whole dataset.
 - ❑ Median income is a continuous numerical attribute, you first need to create an income category attribute
-

-
- ❑ Most **median income values are clustered around 1.5 to 6** (i.e., \$15,000–\$60,000), but some median incomes go far beyond 6.
 - ❑ It is **important to have a sufficient number of instances in your dataset for each stratum**
 - ❑ The following code uses the `pd.cut()` function to create an income category attribute with five categories (labeled from 1 to 5): category 1 ranges from 0 to 1.5 (i.e., less than \$15,000), category 2 from 1.5 to 3, and so on
-

For stratified Splitting

```
import pandas as pd
from sklearn.model_selection import train_test_split

# Load the dataset
url = "https://raw.githubusercontent.com/ageron/handson-ml2/master/datasets/housing/housing.csv"
housing = pd.read_csv(url)

# Separate into features (X) and target (y)
X = housing.drop("median_house_value", axis=1) # Features (all columns except the target)
y = housing["median_house_value"] # Target variable

# Create categories for the target variable
housing["income_cat"] = pd.cut(housing["median_house_value"],
                               bins=[0, 100000, 200000, 300000, 400000, np.inf],
                               labels=[1, 2, 3, 4, 5])

# Split into train and test sets with stratification
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=housing["income_cat"])
```

These income categories are represented in Figure 2-9:

```
housing["income_cat"].hist()
```

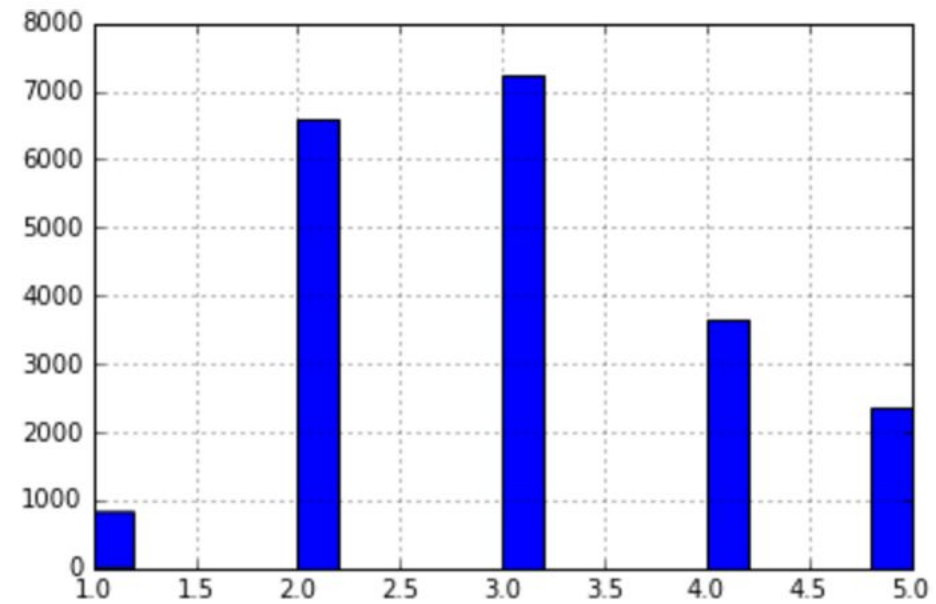


Figure 2-9. Histogram of income categories

Discover and Visualize the Data to Gain Insights

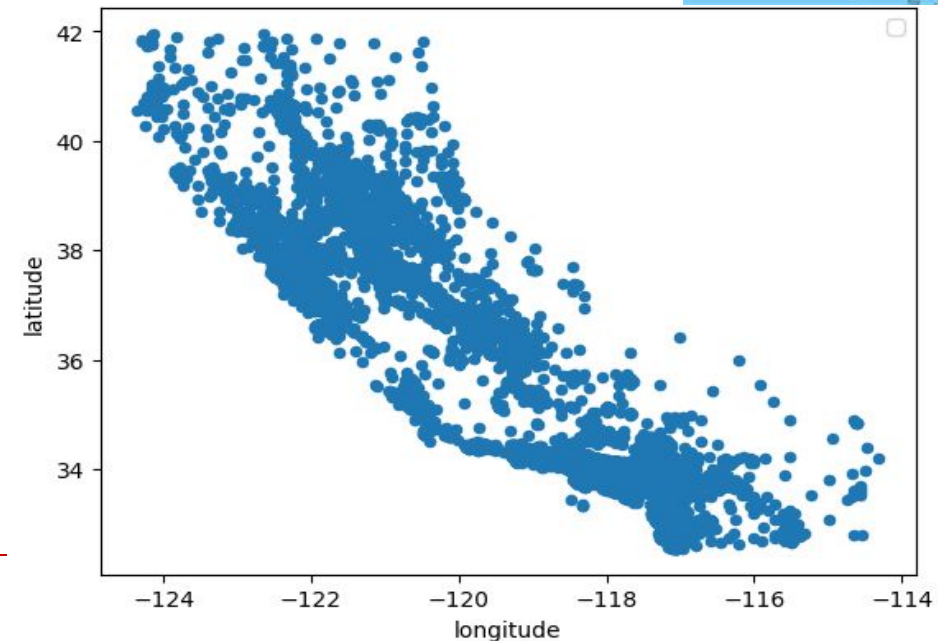
- ❑ So far you have **only taken a quick glance at the data to get a general understanding** of the kind of **data you are manipulating**.
 - ❑ First, make sure you have put the test set aside and you are only exploring the training set.
 - ❑ Since there is geographical information (latitude and longitude), it is a good idea to create a scatterplot of all districts to visualize the data
-

Discover and Visualize the Data to Gain Insights

Visualizing Geographical Data

```
housing.plot(kind="scatter", x="longitude", y="latitude")
```

Geographical scatterplot of the data



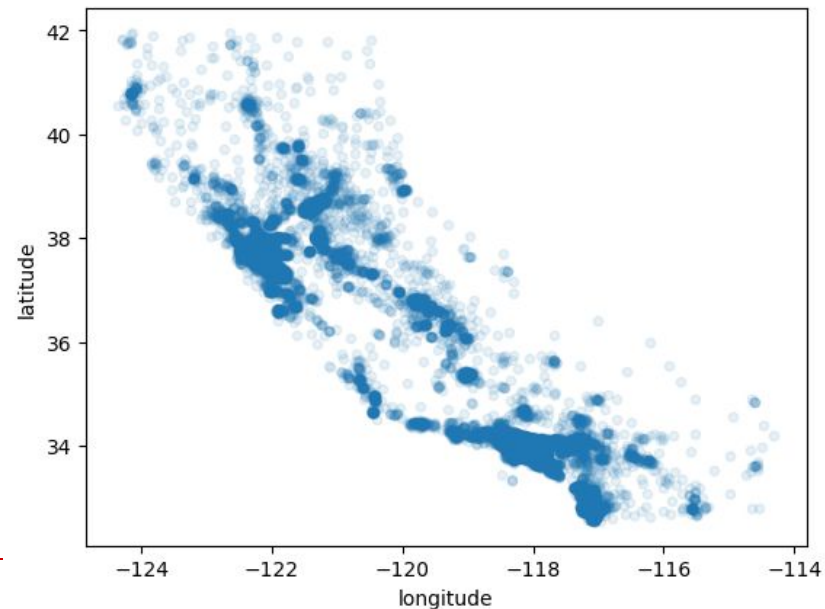
Discover and Visualize the Data to Gain Insights

Visualizing Geographical Data

```
import matplotlib.pyplot as plt
# Add the target variable back to the training set for visualization
train_set = X_train.copy()
train_set["median_house_value"] = y_train
# Plot the training set
train_set.plot(kind="scatter", x="longitude", y="latitude", alpha=0.1)
plt.legend()
```

Setting the alpha option to 0.1 makes it much easier to visualize the places where there is a high density of data points

A better visualization that highlights high-density areas



-
- Now let's look at the housing prices .
 - The radius of each circle represents the district's population (option s), and the color represents the price (option c).
 - Colour ranges from blue (low values) to red (high prices)
-

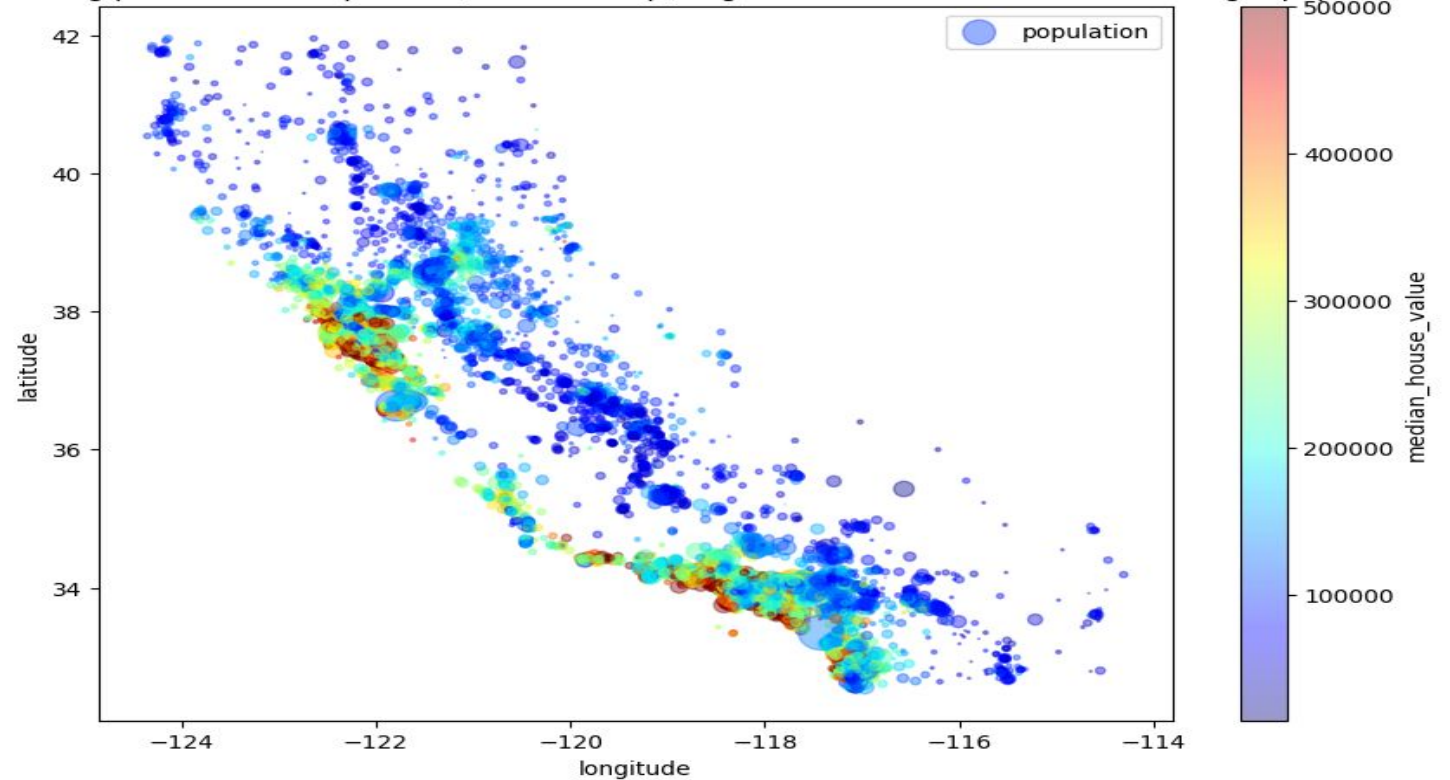
Discover and Visualize the Data to Gain Insights

Visualizing Geographical Data

```
train_set.plot(kind="scatter", x="longitude", y="latitude", alpha=0.4,  
s=train_set["population"]/100, label="population", figsize=(10,7),  
c="median_house_value", cmap=plt.get_cmap("jet"), colorbar=True)
```

This image tells you that the housing prices are very much related to the location (e.g., close to the ocean) and to the population density

California housing prices: red is expensive, blue is cheap, larger circles indicate areas with a larger population



Looking for Correlations

Standard correlation coefficient (also called Pearson's r).

Pearson's correlation coefficient (r) measures the linear relationship between two variables.

It ranges from -1 to 1:

r = 1: Perfect positive linear relationship.

r = -1: Perfect negative linear relationship.

r = 0: No linear relationship.

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \cdot \sum (y_i - \bar{y})^2}}$$

Where:

- x_i and y_i : Individual data points for variables x and y.
 - $\bar{x}$ and $\bar{y}$: Means of x and y.
-

Answer

1. How would you interpret a correlation coefficient of **-0.8**?

A correlation coefficient of **-0.8** indicates a **strong negative linear relationship** between the two variables. This means:

As one variable increases, the other variable tends to **decrease**.

The relationship is **strong** because the absolute value of the correlation coefficient (0.8) is close to 1.

For example, if **x** represents the number of hours spent studying and **y** represents the number of errors in an exam, a correlation of -0.8 would suggest that as study hours increase, the number of errors decreases significantly.

Answer

2. Can Pearson's correlation coefficient capture non-linear relationships? Why or why not?

No, Pearson's correlation coefficient **cannot capture non-linear relationships**. Here's why:

- Pearson's r measures only **linear relationships** between two variables.
- It assumes that the relationship between the variables is linear and that the data is normally distributed.
- If the relationship is non-linear (e.g., quadratic, exponential, or logarithmic), Pearson's r may give a misleading result, often close to 0, even if there is a strong non-linear relationship.

Example:

Consider the following data:

x	y
1	1
2	4
3	9
4	16
5	25

- The relationship between x and y is **quadratic** ($y=x^2$).
 - Pearson's r will not capture this relationship accurately and may give a value close to 0, even though there is a clear non-linear relationship.
-

Correlation Matrix

```
housing = pd.read_csv(url)

# Select only numerical columns (excluding categorical columns like 'ocean_proximity')
numerical_columns = housing.select_dtypes(include=['float64', 'int64'])

# Calculate the correlation matrix
correlation_matrix = numerical_columns.corr()

# Display the correlation of 'median_house_value' with other numerical columns
print(correlation_matrix["median_house_value"].sort_values(ascending=False))
```

Interpretation of the Output

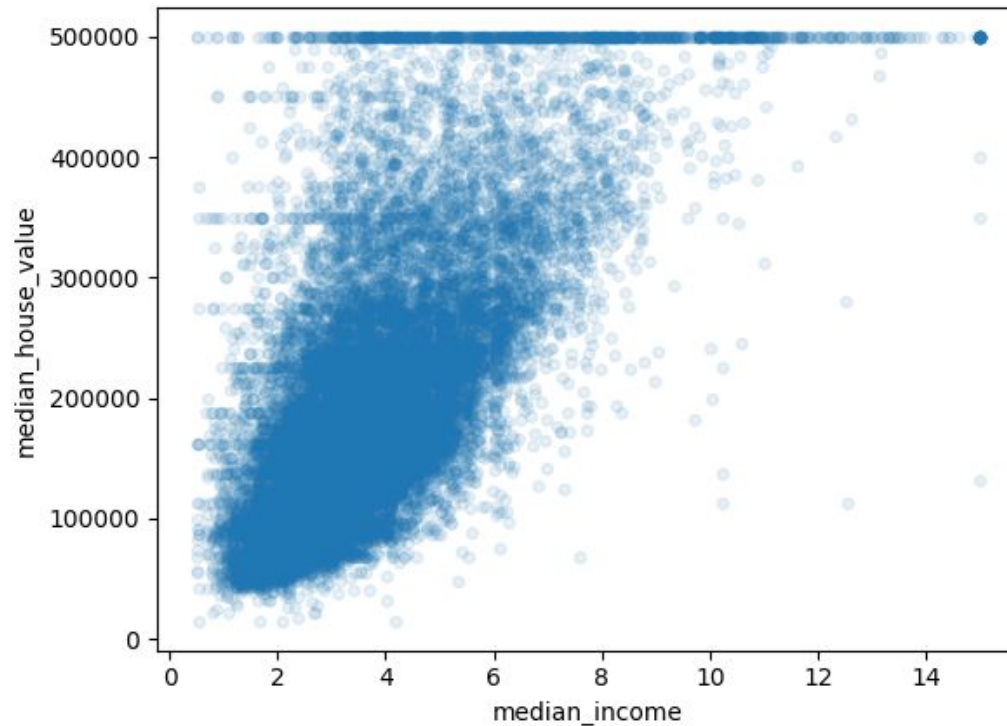
- Each value represents the **Pearson correlation coefficient** between median_house_value and the corresponding numerical column.
- **Key Observations:**
 - **median_income** has the highest positive correlation with **median_house_value** ($r=0.688$), meaning higher income is associated with higher house prices.
 - latitude and longitude have weak negative correlations, indicating that location has a minor impact on house prices in this dataset.
 - total_rooms and housing_median_age have weak positive correlations.

Output

median_house_value	1.000000
median_income	0.688075
total_rooms	0.134153
housing_median_age	0.105623
households	0.065843
total_bedrooms	0.049686
population	-0.024650
longitude	-0.045967
latitude	-0.144160

-
- The most promising attribute to predict the median house value is the median income, so let's zoom in on their correlation scatterplot.

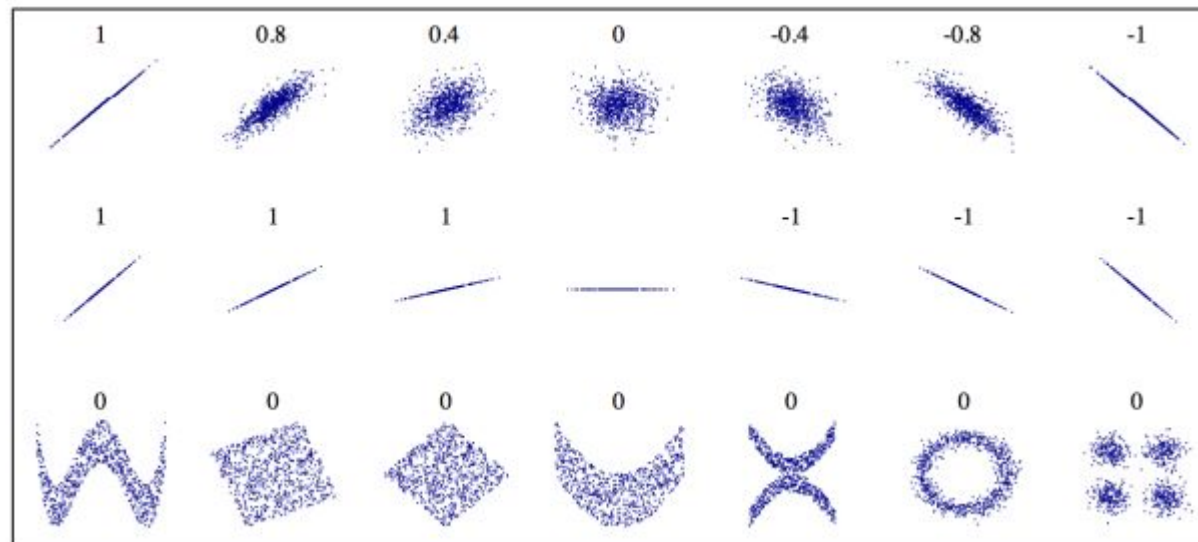
```
housing.plot(kind="scatter", x="median_income", y="median_house_value",  
alpha=0.1)
```



This plot reveals that correlation is indeed very strong; we can clearly see the upward trend, and the points are not too dispersed.

Standard correlation coefficient of various datasets

- The correlation coefficient only measures linear correlations (“if x goes up, then y generally goes up/down”). It may completely miss out on nonlinear relationships (e.g., “if x is close to 0, then y generally goes up”).
- All the plots of the bottom row have a correlation coefficient equal to 0, despite the fact that their axes are clearly not independent: these are examples of nonlinear relationships.
- Also, the second row shows examples where the correlation coefficient is equal to 1 or -1 ; notice that this has nothing to do with the slope. For example, your height in inches has a correlation coefficient of 1 with your height in feet or in nanometers.



Experimenting with Attribute Combinations

- ❑ One last thing you may want to do before preparing the data for Machine Learning algorithms is to try out various attribute combinations.
 - ❑ The **total number of rooms** in a district is not very useful if you **don't know how many households there are**.
 - ❑ the **total number of bedrooms** by itself is not very useful: you probably want to **compare it to the number of rooms**.
 - ❑ And the **population per household** also seems like an interesting attribute combination to look at
-

```
housing["rooms_per_household"] = housing["total_rooms"]/housing["households"]
housing["bedrooms_per_room"] =
housing["total_bedrooms"]/housing["total_rooms"]
housing["population_per_household"]=housing["population"]/housing["households"]
"]
```

And now let's look at the correlation matrix again:

```
>>> corr_matrix = housing.corr()
>>> corr_matrix["median_house_value"].sort_values(ascending=False)
median_house_value      1.000000
median_income           0.687160
rooms_per_household     0.146285
total_rooms             0.135097
housing_median_age      0.114110
households              0.064506
total_bedrooms          0.047689
population_per_household -0.021985
population              -0.026920
longitude               -0.047432
latitude                -0.142724
bedrooms_per_room       -0.259984
Name: median_house_value, dtype: float64
```

The new
bedrooms_per_room
attribute is much
more correlated
with the median
house value than
the total number of
rooms or bedrooms

Prepare the Data for Machine Learning Algorithms

- Let's revert to a clean training set

-
- Since total_bedrooms has missing values, we need to deal with it. Scikit-Learn provides a SimpleImputer instance to take care of missing values.
 - Our numeric attributes are on different scales so we need to apply feature scaling. For standardization, we can use the StandardScaler transformer from Scikit-Learn.
-

Prepare the Data for Machine Learning Algorithms

Data Cleaning:

Most Machine Learning algorithms cannot work with missing features.

To identify which attributes (columns) in a dataset have missing values, we can use the `isnull()` function in pandas combined with `sum()` to count the number of missing values in each column.

```
# Check for missing values in each column  
missing_values = housing.isnull().sum()
```

```
# Display columns with missing values  
print(missing_values[missing_values > 0])
```

Output

```
total_bedrooms 207
```

Prepare the Data for Machine Learning Algorithms

Data Cleaning:

total_bedrooms attribute has some missing values, to fix it, we have three options:

1. Get rid of the corresponding districts.
2. Get rid of the whole attribute.
3. Set the values to some value (zero, the mean, the median, etc.).

```
housing_copy=housing
```

```
#Option-1: Get rid of the corresponding districts.
```

```
housing_copy.dropna(subset=["total_bedrooms"])
```

```
#Option-2: Get rid of the whole attribute.
```

```
housing_copy.drop("total_bedrooms", axis=1)
```

Data Cleaning

Filling the missing values by setting them to a specific value like zero, the mean, or the median, **a process called "imputation"**

1. Mean Imputation: **Replace missing values with the average value of the column where the missing data occurs.**
2. Median Imputation: Replace missing values with the middle value of the column, which is less sensitive to outliers compared to the mean.
3. Mode Imputation: For categorical data, replace missing values with the most frequent value in the column.
4. Constant Value Imputation: Assign a specific constant value (like zero) to all missing values.

Important considerations when choosing a method:

Data Distribution:

1. Normal distribution: Use the mean imputation.
 2. Skewed distribution: Use the median imputation to mitigate the impact of outliers.
-

Data Cleaning

Using `SimpleImputer` from `scikit-learn` is a more structured and scalable way to fill missing values in a `DataFrame`.

```
import pandas as pd
import numpy as np

# Create a DataFrame with missing values
data = { "Age": [25, 30, np.nan, 35, 40, np.nan],
          "Salary": [50000, np.nan, 60000, np.nan, 70000, 80000] }
df = pd.DataFrame(data)
print("Original DataFrame:")
print(df)
```

```
Original DataFrame:
   Age  Salary
0  25.0  50000.0
1  30.0     NaN
2   NaN  60000.0
3  35.0     NaN
4  40.0  70000.0
5   NaN  80000.0
```

Data Cleaning

1. Filling Missing Values with Mean

```
from sklearn.impute import SimpleImputer
# Create an imputer with the mean strategy
imputer_mean = SimpleImputer(strategy="mean")

# Fit and transform the DataFrame
df_mean = pd.DataFrame(imputer_mean.fit_transform(df), columns=df.columns)
print("\nDataFrame after filling missing values with mean:")
print(df_mean)
```

```
DataFrame after filling missing values with mean:
   Age  Salary
0  25.0  50000.0
1  30.0  65000.0
2  32.5  60000.0
3  35.0  65000.0
4  40.0  70000.0
5  32.5  80000.0
```

Data Cleaning

Filling Missing Values with Median

Create an imputer with the median strategy

```
imputer_median = SimpleImputer(strategy="median")
```

Fit and transform the DataFrame df_median =

```
pd.DataFrame(imputer_median.fit_transform(df), columns=df.columns)
```

```
print("\nDataFrame after filling missing values with median:") print(df_median)
```

```
DataFrame after filling missing values with median:
   Age  Salary
0  25.0  50000.0
1  30.0  65000.0
2  32.5  60000.0
3  35.0  65000.0
4  40.0  70000.0
5  32.5  80000.0
```


Data Cleaning

Question

- ☐ How would you handle missing values in a categorical column using SimpleImputer?

Data Cleaning

Handling Missing Values in a Categorical Column

```
# Create a DataFrame with missing categorical values
```

```
data = { "Name": ["Alice", "Bob", "Charlie", "David", "Eva", np.nan], "City": ["New York", np.nan, "Chicago", "Houston", np.nan, "Los Angeles"] }
```

```
df = pd.DataFrame(data)
```

```
print("Original DataFrame:")
```

```
print(df)
```

Original DataFrame:

	Name	City
0	Alice	New York
1	Bob	NaN
2	Charlie	Chicago
3	David	Houston
4	Eva	NaN
5	NaN	Los Angeles

Data Cleaning

To handle missing values in a **categorical column** using **SimpleImputer**, you can use the **strategy="most_frequent"** option. This strategy fills missing values with the **mode** (the most frequently occurring value) of the column.

```
from sklearn.impute import SimpleImputer
# Create an imputer with the "most_frequent" strategy
imputer = SimpleImputer(strategy="most_frequent")
# Fit and transform the DataFrame
df_imputed = pd.DataFrame(imputer.fit_transform(df), columns=df.columns)
print("\nDataFrame after filling missing values:")
print(df_imputed)
```

```
DataFrame after filling missing values:
   Name      City
0  Alice  New York
1   Bob   New York
2 Charlie  Chicago
3  David   Houston
4   Eva   New York
5  Alice Los Angeles
```

-
- Since total_bedrooms has missing values, we need to deal with it. Scikit-Learn provides a SimpleImputer instance to take care of missing values.
-

Data Cleaning

To fill missing values for the `total_bedrooms` attribute using the **median** strategy #Option3: Set the values to some value (zero, the mean, the median, etc.).

Step 1: Create an instance of SimpleImputer with the median strategy

```
imputer = SimpleImputer(strategy="median")
```

```
housing_copy=housing
```

Step 2: Fit the imputer on the "total_bedrooms" column

Note: SimpleImputer expects a 2D array, so we reshape the column

```
imputer.fit(housing_copy[["total_bedrooms"]])
```

Step 3: Transform (fill) the missing values in the "total_bedrooms" column

```
housing_copy["total_bedrooms"] = imputer.transform(housing[["total_bedrooms"]])
```

Verify that there are no missing values left

```
print(housing_copy["total_bedrooms"].isnull().sum())
```

Data Cleaning

Explanation of the Code

1. **SimpleImputer:**

1. This is a scikit-learn class used to fill missing values.
2. The `strategy="median"` parameter specifies that missing values should be replaced with the median of the column.

2. **fit():** The `fit()` method calculates the median of the `total_bedrooms` column (ignoring missing values).

3. **transform():** The `transform()` method replaces the missing values in the `total_bedrooms` column with the calculated median.

4. **Reshape to 2D Array:** `SimpleImputer` expects a 2D array (a `DataFrame` or a 2D `NumPy` array), so we pass `housing[["total_bedrooms"]]` (double brackets) instead of `housing["total_bedrooms"]`.

5. **Verification:** After filling the missing values, we check if there are any remaining missing values using `isnull().sum()`.

Handling Text and Categorical Attributes

- **Categorical Data:** Data that represents categories or labels (e.g., colors, sizes, cities).
 - **Types of Categorical Data:**
 - **Ordinal:** Categories have a natural order (e.g., small, medium, large).
 - **Nominal:** Categories have no natural order (e.g., red, blue, green).
-

Handling Text and Categorical Attributes

- ❑ **Encoding Categorical Data**
 - ❑ Machine learning algorithms **require numerical input**, so categorical data must be converted into numerical form.
 - ❑ Two common methods are:
 - 1. Ordinal Encoding.**
 - 2. One-Hot Encoding.**
-

Handling Text and Categorical Attributes

What is Ordinal Encoding?

- Assigns a unique integer to each category based on its order (for ordinal data).
- Example: small = 0, medium = 1, large = 2

Handling Text and Categorical Attributes

```
import pandas as pd
from sklearn.preprocessing import OrdinalEncoder

# Example dataset
data = {
    "Size": ["small", "medium", "large", "medium", "small"]
}
df = pd.DataFrame(data)

# Initialize OrdinalEncoder
ordinal_encoder = OrdinalEncoder(categories=[["small", "medium", "large"]])

# Fit and transform the data
df["Size_Encoded"] = ordinal_encoder.fit_transform(df[["Size"]])
print(df)
```

	Size	Size_Encoded
0	small	0.0
1	medium	1.0
2	large	2.0
3	medium	1.0
4	small	0.0

Handling Text and Categorical Attributes

□ What is One-Hot Encoding?

- Creates binary columns for each category.
- Example: small = [1, 0, 0], medium = [0, 1, 0], large = [0, 0, 1].

Handling Text and Categorical Attributes

```
from sklearn.preprocessing import OneHotEncoder
# Example dataset
data = {
    "Color": ["red", "blue", "green", "blue", "red"]
}
df = pd.DataFrame(data)

# Initialize OneHotEncoder
onehot_encoder = OneHotEncoder(sparse=False)

# Fit and transform the data
encoded_data = onehot_encoder.fit_transform(df[["Color"]])

# Convert to DataFrame for better visualization
encoded_df = pd.DataFrame(encoded_data, columns=onehot_encoder.get_feature_names_out(["Color"]))
df_encoded = pd.concat([df, encoded_df], axis=1)
print(df_encoded)
```

	Color	Color_blue	Color_green	Color_red
0	red	0.0	0.0	1.0
1	blue	1.0	0.0	0.0
2	green	0.0	1.0	0.0
3	blue	1.0	0.0	0.0
4	red	0.0	0.0	1.0

Handling Text and Categorical Attributes

Method	OrdinalEncoder	OneHotEncoder
Use Case	Ordinal data (natural order)	Nominal data (no natural order)
Output	Single column with integer values	Multiple binary columns (one per category)
Advantages	Preserves order, simple, efficient	No order assumption, avoids bias
Disadvantages	Not suitable for nominal data	Increases dimensionality

```
from sklearn.preprocessing import OneHotEncoder
```

```
# fetch ocean_proximity
housing_cat = housing[["ocean_proximity"]]
# one hot encoder
cat_encoder = OneHotEncoder()
housing_cat_encoder = cat_encoder.fit_transform(housing_cat)
# convert sparse matrix to array to display the result
result = housing_cat_encoder.toarray()
print("Data:")
print(result)
category_list = cat_encoder.categories_
print("Category List::", category_list)
```

```
Data:
[[0. 0. 0. 1. 0.]
 [0. 0. 0. 1. 0.]
 [0. 0. 0. 1. 0.]
 ...
 [0. 1. 0. 0. 0.]
 [0. 1. 0. 0. 0.]
 [0. 1. 0. 0. 0.]]
Category List:: [array(['<1H OCEAN', 'INLAND', 'ISLAND', 'NEAR BAY', 'NEAR OCEAN'],
      dtype=object)]
```

Representation learning

- If a categorical attribute has a **large number of possible categories** (e.g., country code, profession, species), then one-hot encoding will **result in a large number of input features**. This may slow down training and degrade performance.
 - If this happens, you may want **to replace the categorical input with useful numerical features related to the categories**:
 - for example, you could replace the **ocean_proximity** feature with the **distance to the ocean** (similarly, a country code could be replaced with the country's population and GDP per capita).
 - Alternatively, you could **replace each category with a learnable, low-dimensional vector called an embedding**. Each category's representation would be **learned during training**. This is an example of representation learning
-

Handling Outliers in dataset

What is Outlier ?

An outlier is a data point significantly different from other data points in a dataset.

Example: Consider a small dataset, sample= [15, 101, 18, 7, 13, 16, 11, 21, 5, 15, 10, 9]. By looking at it, one can quickly say '101' is an outlier that is much larger than the other values.

+-----+-----+	
with outlier	without outlier
+-----+-----+	
Mean: 20.08	Mean: 12.72
Median: 14.0	Median: 13.0
Mode: 15	Mode: 15
Variance: 614.74	Variance: 21.28
Std dev: 24.79	Std dev: 4.61
+-----+-----+	



From the above calculations, we can clearly say the Mean is more affected than the Median.

Detecting Outliers

If our dataset is small, we can detect the **outlier** by just looking at the dataset. But what if we have a huge dataset, how do we identify the outliers then? We need to use visualization and mathematical techniques.

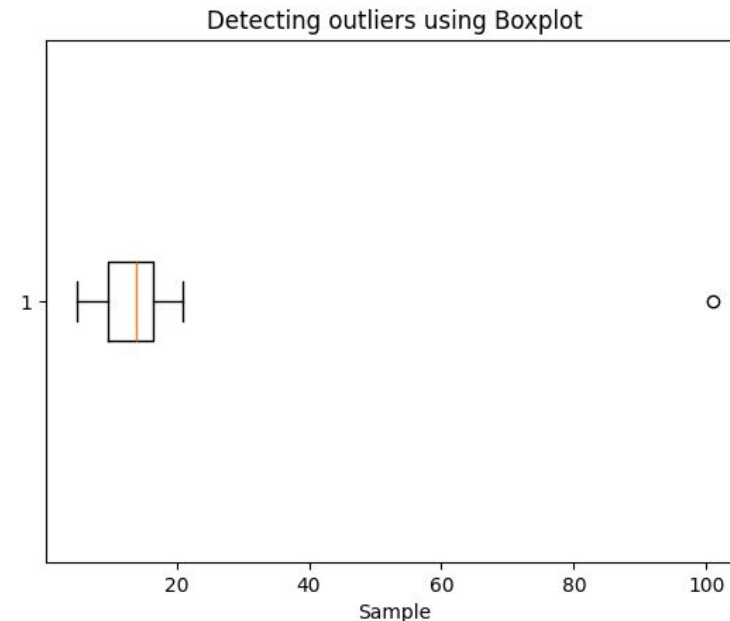
Below are some of the techniques of detecting outliers

- Boxplots
- Z-score
- Inter Quantile Range(IQR)

Detecting Outliers Using Boxplot

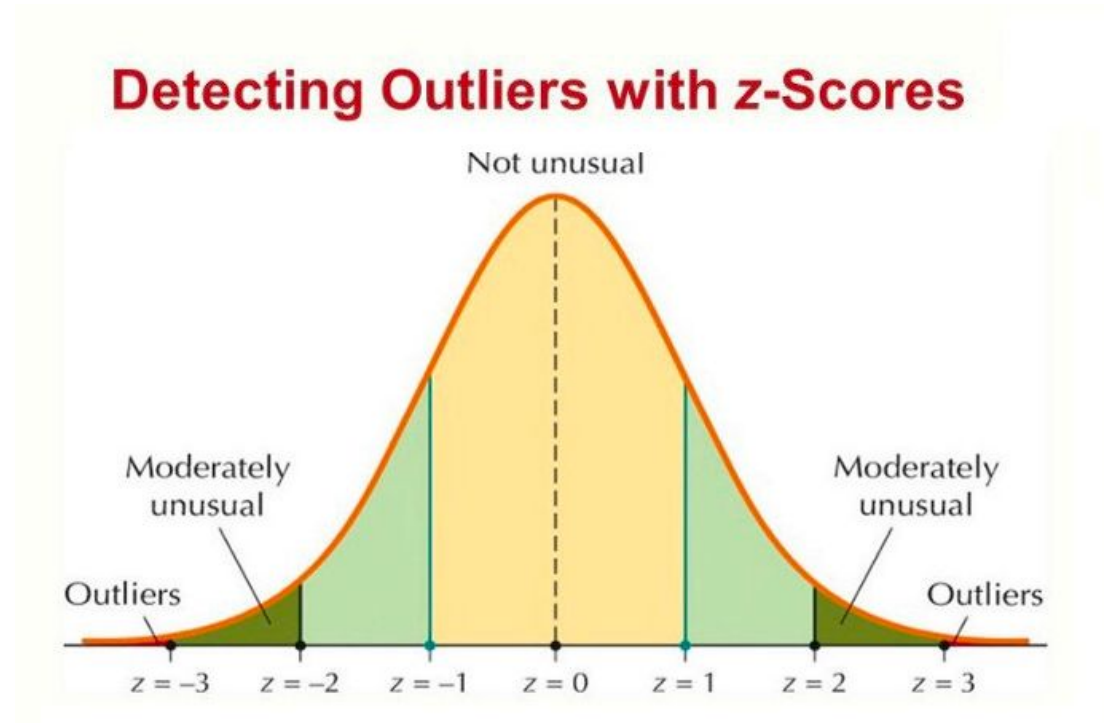
To visualize outliers in a dataset using a boxplot, you can plot your data on a boxplot, where data points that fall significantly outside the "whiskers" of the box are considered outliers; these outliers will be displayed as individual points beyond the box, allowing you to easily identify them visually.

```
import matplotlib.pyplot as plt
sample= [15, 101, 18, 7, 13, 16, 11, 21, 5, 15, 10, 9]
plt.boxplot(sample, vert=False)
plt.title("Detecting outliers using Boxplot")
plt.xlabel('Sample')
plt.show()
```



Detecting Outliers using the Z-scores

Any data point whose Z-score falls out of 3rd standard deviation is an outlier treatment.



Detecting Outliers using the Z-scores

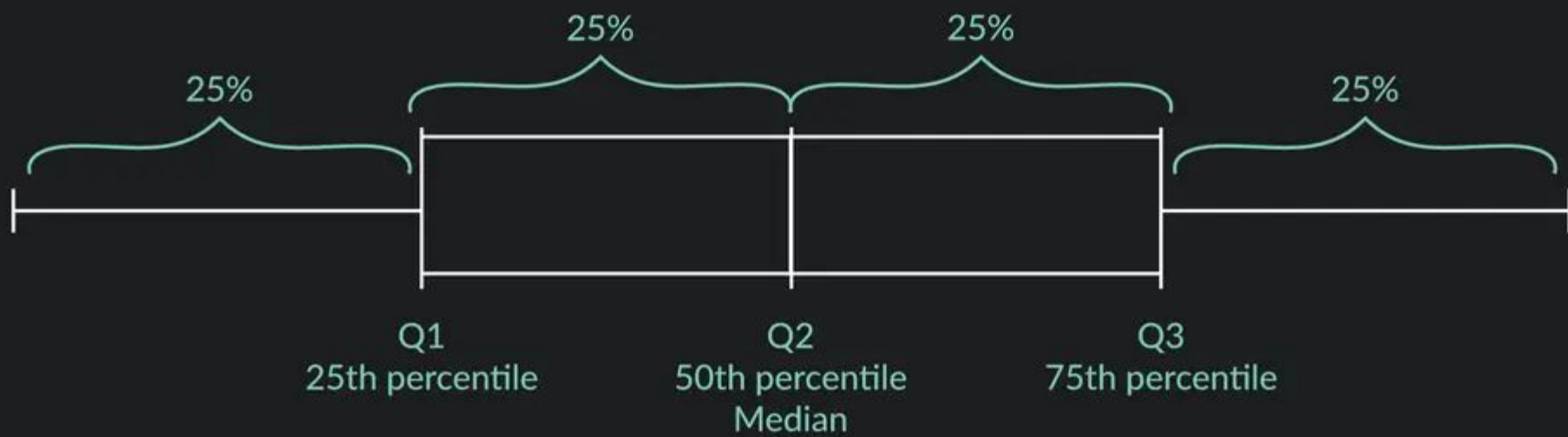
Steps:

- loop through all the data points and compute the Z-score using the formula $(X_i - \text{mean}) / \text{std}$.
- define a threshold value of 3 and mark the datapoints whose absolute value of Z-score is greater than the threshold as outliers.

Detecting Outliers using the Z-scores

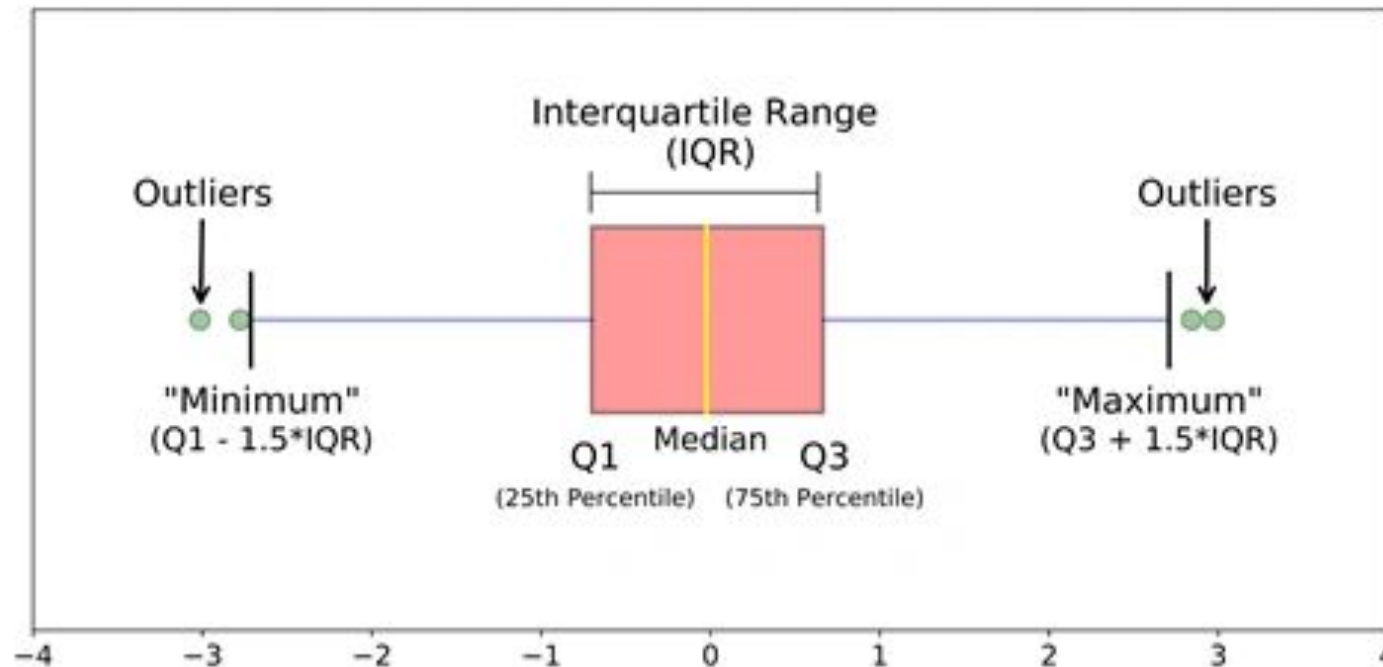
```
import numpy as np
outliers = []
def detect_outliers_zscore(data):
    thres = 3
    mean = np.mean(data)
    std = np.std(data)
    # print(mean, std)
    for i in data:
        z_score = (i-mean)/std
        if (np.abs(z_score) > thres):
            outliers.append(i)
    return outliers
sample_outliers = detect_outliers_zscore(sample)
print("Outliers from Z-scores method: ", sample_outliers)
```

**Outliers from Z-scores
method:
[101]**



Detecting Outliers using the Inter Quantile Range(IQR)

Criteria: Data points that lie 1.5 times of IQR above Q3 and below Q1 are outliers.



Detecting Outliers using the Inter Quantile Range(IQR)

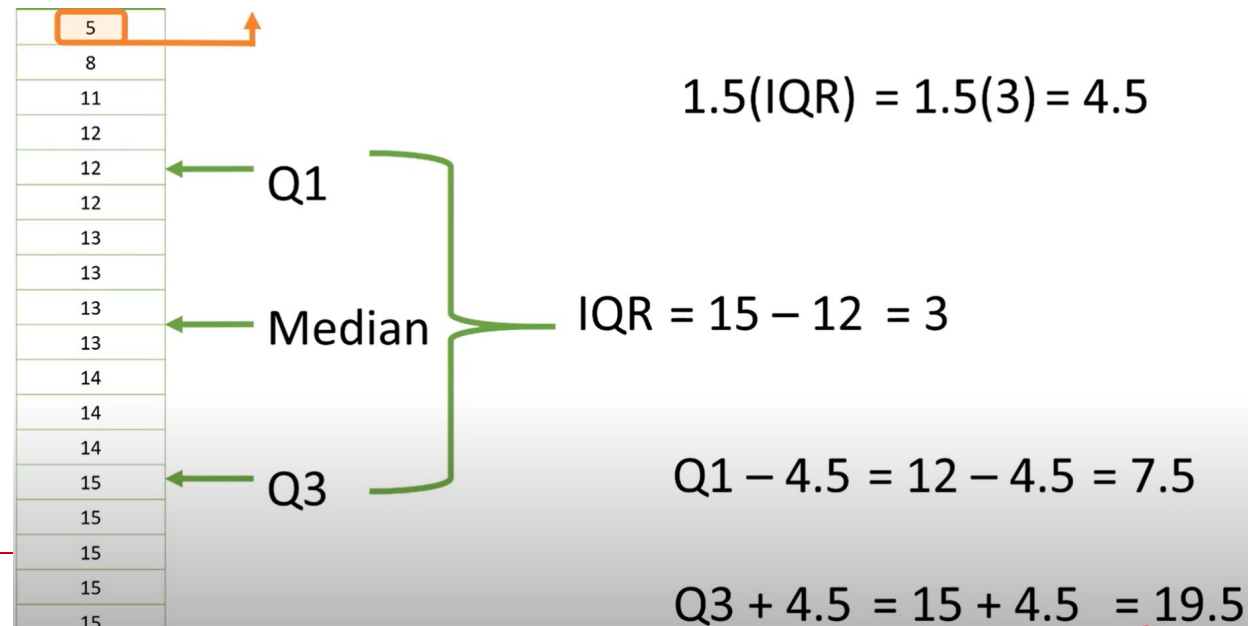
Steps

- Sort the dataset in ascending order
 - calculate the 1st and 3rd quartiles($Q1$, $Q3$)
 - compute $IQR = Q3 - Q1$
 - compute lower bound = $(Q1 - 1.5 * IQR)$, upper bound = $(Q3 + 1.5 * IQR)$
 - loop through the values of the dataset and check for those who fall below the lower bound and above the upper bound and mark them as outlier treatment in python
-

Detecting Outliers using the Inter Quantile Range(IQR)

Steps

- Sort the dataset in ascending order
- calculate the 1st and 3rd quartiles(Q1, Q3)
- compute $IQR = Q3 - Q1$
- compute lower bound = $(Q1 - 1.5 * IQR)$, upper bound = $(Q3 + 1.5 * IQR)$
- loop through the values of the dataset and check for those who fall below the lower bound and above the upper bound and mark them as outlier treatment in python



Detecting Outliers using the Inter Quantile Range(IQR)

```
outliers = []
def detect_outliers_iqr(data):
    data = sorted(data)
    q1 = np.percentile(data, 25)
    q3 = np.percentile(data, 75)
    # print(q1, q3)
    IQR = q3-q1
    lwr_bound = q1-(1.5*IQR)
    upr_bound = q3+(1.5*IQR)
    # print(lwr_bound, upr_bound)
    for i in data:
        if (i<lwr_bound or i>upr_bound):
            outliers.append(i)
    return outliers

sample_outliers = detect_outliers_iqr(sample)
print("Outliers from IQR method: ", sample_outliers)
```

Feature Scaling

- ❑ One of the most important transformations you need to apply to your data is feature scaling.
 - ❑ Machine Learning algorithms don't perform well when the input numerical attributes have very different scales.
 - ❑ This is the case for the housing data: the total number of rooms ranges from about 6 to 39,320, while the median incomes only range from 0 to 15.
 - ❑ There are two common ways to get all attributes to have the same scale: min max scaling and standardization.
-

-
- Min-max scaling (many people call this normalization) : values are shifted and rescaled so that they end up ranging from 0 to 1.
 - Standardization is different: first it subtracts the mean value, and then it divides by the standard deviation so that the resulting distribution has unit variance
-

MinMax Scaler

- MinMax Scaler is one of the most popular scaling algorithms. It transforms features by scaling each feature to a given range, which is generally $[0,1]$, or $[-1,-1]$ in case of negative values.
- For each feature, the MinMax Scaler follows the formula:

$$\frac{x_i - \min(x)}{\max(x) - \min(x)}$$

- It subtracts the mean of the column from each value and then divides by the range, i.e, $\max(x)-\min(x)$.
 - This scaling algorithm works very well in cases where the standard deviation is very small, or in cases which don't have Gaussian distribution.
-

MinMax Scaler

Age		Normalized Age
44		0.80952381
27		0
30	Normalization	0.142857143
38	→	0.523809524
40		0.619047619
35		0.380952381
48		1

Range 0-1

How to calculate Normalized value?

$X = 35$, $\min = 27$, $\max = 48$ for column Age.

$X_{\text{norm}}(\text{for } 35) = \frac{35-27}{48-27} = 0.3809$

$$\frac{x_i - \min(x)}{\max(x) - \min(x)}$$

Standard Scaler

- Standard Scaler follows the Standard Normal Distribution, i.e., it assumes a normal distribution for data within each feature. The scaling makes the distribution centered around 0, with a standard deviation of 1 and the mean removed.
- For a data sample x , its score for Standard Scaler is calculated as:

$$\frac{x_i - \text{mean}(x)}{sd(x)}$$

- Where sd is the standard deviation of x .
-

Standard Scaler

Age		Age	Standardized Age
44		44	0.954611636
27		27	-1.514927162
30		30	-1.079126198
38		38	0.083009708
40		40	0.373543684
35		35	-0.352791257
48		48	1.535679589
Mean = 37.42857			
Std. Dev. 6.883871			

Standardization

How to calculate Standardized value?
X = 35, mean = 37.42, Std. Dev. = 6.88
for column Age.
 $X_{std}(\text{for } 35) = \frac{35 - 37.42}{6.88} = -0.3527$

Mean = 0
Std. dev. = 1

$$\frac{x_i - \text{mean}(x)}{sd(x)}$$

Transformation Pipeline

- There are many data transformation steps that need to be executed in the right order.
- Fortunately, Scikit-Learn provides the Pipeline class to help with such sequences of transformation

```
from sklearn.compose import ColumnTransformer

num_attribs = list(housing_num)
cat_attribs = ["ocean_proximity"]

full_pipeline = ColumnTransformer([
    ("num", num_pipeline, num_attribs),
    ("cat", OneHotEncoder(), cat_attribs),
])

housing_prepared = full_pipeline.fit_transform(housing)
```

Select and Train a Model

- ❑ Framed the problem,
 - ❑ got the data and explored it,
 - ❑ sampled a training set and a test set,
 - ❑ wrote transformation pipelines to clean up and prepare your data for Machine Learning algorithms automatically.
-
- ❑ You are now ready to select and train a Machine Learning model
-

Training and Evaluating on the Training Set

□ Train a Linear Regression model

```
from sklearn.linear_model import LinearRegression  
  
lin_reg = LinearRegression()  
lin_reg.fit(housing_prepared, housing_labels)
```

-
- ❑ `print("Predictions:", lin_reg.predict(some_data_prepared))`
 - ❑ Predictions: [210644.6045 317768.8069
210956.4333 59218.9888 189747.5584]

`print("Labels:", list(some_labels))` Labels: [286600.0,
340600.0, 196900.0, 46300.0, 254500.0]

the predictions are not exactly accurate (e.g., the first prediction is off by close to 40%)

Metrics to evaluate the Regression Models

Common metrics used to evaluate regression models in machine learning include:

1. Mean Absolute Error (MAE)
 2. Root Mean Squared Error (RMSE)
 3. Mean Squared Error (MSE)
-

Metrics to evaluate the Regression Models

Mean Absolute Error (MAE)

It's a measurement of the typical absolute discrepancies between a dataset's actual values and predicted values.

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

where

y_i = actual value

$\hat{y}_i$ = predicted value

n = sample size

Metrics to evaluate the Regression Models

Root Mean Squared Error (RMSE):

It quantifies the differences between predicted values and actual values, squaring the errors, taking the mean, and then finding the square root.

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Metrics to evaluate the Regression Models

Mean Squared Error (MSE):

It measures the square root of the average discrepancies between a dataset's actual values and predicted values.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Question

Consider the following training set Table 5.4 for predicting the sales of the items.

Table 5.4: Training Item Table

Items x_i	Actual Sales (In Thousands) y_i
I_1	80
I_2	90
I_3	100
I_4	110
I_5	120

Consider two fresh items I_6 and I_7 , whose actual values are 80 and 75, respectively. A regression model predicts the values of the items I_6 and I_7 as 75 and 85, respectively. Find MAE, MSE, RMSE,

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad \text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad \text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Solution

Solution: The test items' actual and prediction is given in Table 5.5 as:

Table 5.5: Test Item Table

Test Items	Actual Value y_i	Predicted Value $\bar{y}_i$
I_6	80	75
I_7	75	85

Mean Absolute Error (MAE) using Eq. (5.12) is given as:

$$\text{MAE} = \frac{1}{2} \times |80 - 75| + |75 - 85| = \frac{15}{2} = 7.5$$

Mean Squared Error (MSE) using Eq. (5.13) is given as:

$$\text{MSE} = \frac{1}{2} \times |80 - 75|^2 + |75 - 85|^2 = \frac{125}{2} = 62.5$$

Root Mean Square error using Eq. (5.14) is given as:

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{62.5} = 7.91$$

Table 5.4: Training Item Table

Items x_i	Actual Sales (In Thousands) y_i
I_1	80
I_2	90
I_3	100
I_4	110
I_5	120

Thanks for Listening
